



EINFÜHRUNG IN AI

Die Sammlung



V1.02

©

Inhaltsverzeichnis

Einführung in AI (Folien).....	7
Modul 1: Grundlagen, Ausrichtungen, Anwendungen	7
1. Definitionen und Überblick über KI-Ausrichtungen	7
Definition	7
Künstliche Intelligenz (KI)	7
Maschinelles Lernen (ML).....	7
Die KI-Apotheke	7
Auswahl einer KI-Methode	7
Turing-Test und Chatbots	7
2. Geschichtliche Entwicklung	8
Ausgewählte Meilensteine der KI.....	8
Überblick über die KI Entwicklung	8
Entwicklung und Trends	9
3. Gesellschaft	9
Gesellschaftliche Aspekte	9
KI und Verkehr	9
Servicerobotik.....	9
4. Technologien	10
Ausrichtungen von KI	10
Logik-basierte KI	10
Wissensrepräsentation	10
Technologie – Wissensbasierte Systeme	10
Suche.....	10
Planung.....	11
Heuristik.....	11
Evolutionäre Programmierung	11
Maschinelles Lernen & Data Mining	11
Technologie – Agenten	11
5. Anwendungsbereiche	12
Maschinelles Sehen (Bildanalyse)	12
Linguistische Datenverarbeitung.....	12
Spracherkennung	12
Expertensysteme.....	12
Spiele	13

Robotik	13
Eckpunkte des Moduls	13
Modul 2: Logik	14
1. Aussagenlogik	14
Was versteht man unter Aussagenlogik?	14
Syntax.....	14
Möglichkeiten dieser Syntax	14
Semantik	14
Wahrheitstabelle für logische Operatoren	15
Äquivalenz – Syntax vs. Semantik	15
In der Praxis wichtige Äquivalenzen.....	16
Die Folgerung (Schließen, Inferenz)	16
Berechenbarkeit / Komplexität	16
Anwendungen	16
2. Prädikatenlogik	17
Prädikatenlogik: Mächtiger Lösungsansatz	17
Motivation	17
Beispiele	17
Beispiel: Stammbaum.....	18
Anwendungen	18
3. Logikorientierte Programmierung.....	18
Konventionen	19
Terminologie	19
Komponenten	19
Intuition	19
Beispiele	20
Beispiel: Reachability.....	20
Beispiel: Familienstammbaum (nur IDB).....	20
Logikprogrammiersysteme.....	20
Logikprogrammierung nach der Answer Set Semantik.....	20
System clingo	21
Eckpunkte des Moduls	21
Modul 3: Maschinelles Lernen.....	22
1. Überblick Machine Learning	22
Was ist maschinelles Lernen?	22
Konzept des maschinellen Lernens	22

Workflow für prädikative Methoden	22
Definition der Datenpunkte	23
Typische Machine Learning Aufgaben	23
Überwachtes Lernen	23
Unüberwachtes Lernen	23
Beispiel für Klassifikation	23
Maschinelles Lernen	24
2. Überwachtes Lernen	24
Nearest Neighbor (NN)	24
Analyse von NN mittels Voronoi Diagramm	24
k-Nearest Neighbor (kNN)	24
Erweiterung (k)NN für mehrere Klassen	25
Lernen von Entscheidungsbäumen	25
Klassifikationsbaum: Definition, Beispiel	25
Entscheidungsbaum: Definition, Beispiel	25
Pruning – der Baumschnitt	26
Qualitätsbetrachtungen	26
Gütekriterien von (supervised) ML-Verfahren	26
Wahrheitsmatrix (Confusion Matrix)	27
Sensitivität (true/positive)	27
Falsch-negativ-Rate	27
Korrekt- und Falschklassifikationsrate	27
3. Unüberwachtes Lernen	28
Clustering	28
k-Means	28
k-Means mit $k=2$	28
Bewertung eines Clusterings	28
Silhouette-Width	29
OMRk-Algorithmus	29
Beispiel: OMRk-Algorithmus	30
Eckpunkte des Moduls	30
Modul 4: Neuronale Netze	31
1. Grundlagen	31
Grundlegende Idee	31
Mathematisches Modell (Neuron)	31

Mathematisches Modell (Neuronales Netz)	31
Mathematisches Modell	32
2. Das Perzepton	32
Linearer Klassifizierer (Perzeptron)	32
Lernregel für das Perzeptron	33
Beispiel	33
Optimierung und Ausblick	33
3. Mehrlagige Neuronale Netze (Feedforward, Gradientenabstieg und Backpropagation)	34
Lineare Netze mit minimalem Fehler	34
Einblicke in die Mathematik dahinter	34
Inkrementelles Lernen	35
Parameter η der Delta-Regel	35
Backpropagation-Agorithmus	35
Idee	36
Anwendungsbeispiel: NETtalk	36
Software rund um Neuronale Netze	37
4. Deep Learning und Anwendungen	37
Fluch der Dimension	37
Mögliche Ansätze	37
Deep Learning	37
Convolution Neural Network (CNN)	38
Deep Learning in der Praxis	38
Anwendungen – Kreativität	38
Anwendung: ChatGPT (und ähnliche LLMs)	39
Fazit: Neuronale Netze	40
5. Reinforcement Learning und Anwendungen	40
Reinforcement Learning	40
Demo	41
Eckpunkte des Moduls	41
Übungsaufgaben/Alte Tests	42
Abschlussprüfung Haupttermin 2023	42
1. Grundlagen (24 Punkte)	42
a) Nennen Sie 3 Anwendungen der AI und beschreiben Sie diese kurz.	42
b) Erklären Sie den Unterschied der Begriffe „künstliche Intelligenz“, „Machine Learning“ und „Deep Learning“.	42

c) Worum geht es beim sogenannten Turing-Test?	43
2. Multiple-Choice (15 Punkte)	43
3. Logik (12 Punkte).....	43
Angabe	43
Lösung.....	44
4. Maschinelles Lernen (12 Punkte)	44
1. Skizzieren Sie für folgende Punktmenge eines kNN-Modells ($k=1$) das Voronio-Diagramm ein. Die Daten sind als „+“ und „-“ gegeben. (6 Punkte).....	44
2. Wie wird gemäß diesem Modell ($k=1$) der Datenpunkt „?“ klassifiziert? Warum? (3 Punkte)	44
3. Wie würde der mit „?“ markierte Datenpunkt bei $k=2$ klassifiziert? Warum? (3 Punkte)	45
5. Neuronale Netze (12 Punkte)	45
a) Wie ist ein neuronales Netz aufgebaut?	45
b) Welche Idee steckt hinter dem Konzept Backpropagation?	45
c) Welche Probleme können beim Trainieren von neuronalen Netzen auftreten und welche Möglichkeiten gibt es, diese zu bewältigen?	45
d) Welche Aufgaben haben die Faltungs- und Pooling-Schichten in der Topologie eines CNN Netzes?.....	46
Fragenkatalog „Einführung AI“	46
Modul 1: Grundlagen, Ausrichtungen, Anwendungen	46
1. Nennen Sie 3 Anwendungen der künstlichen Intelligenz und beschreiben Sie diese kurz. 46	
2. Nennen Sie 3 Methoden der künstlichen Intelligenz und beschreiben Sie diese kurz. 46	
3. Erklären Sie den Unterschied zwischen den Begriffen „künstliche Intelligenz“, „Machine Learning“ und „Deep Learning“.	46
4. Nennen Sie zwei AI-Methoden und ordnen Sie jeder davon eine typische Anwendung zu, die mit der jeweils anderen Methode nicht gut gelöst werden kann.	46
5. Worum geht es beim sogenannten Turing-Test?	46
Modul 2: Logik.....	47
1. Was versteht man in der Logik unter einer Schlussfolgerung?	47
2. Zeigen Sie, dass folgende Formeln Tautologien sind:	47
3. Formalisieren Sie folgende logische Funktionen mit logischen Operatoren und zeigen Sie, dass Ihre Formel korrekt ist:	47
4. Formalisieren Sie folgende Aussagen in der Prädikatenlogik:	48
5. Schreiben Sie ein Clingo-Programm (vgl. Moodle-Aktivitäten zum Thema Logikprogrammierung) für folgende Aufgabe:.....	48

Modul 3: Maschinelles Lernen	49
1. Erklären Sie den Unterschied zwischen supervised und supervised learning	49
2. Was versteht man unter Occam's Razor und warum sollte man es anwenden?	49
3. Was versteht man bei Klassifikationsaufgaben unter „false positives“ und „false negatives“?	49
4. k-Nearest Neighbor.....	49
5. Gegeben sind die folgenden Punkte im 2-dimensionalen Raum: p1(40/53), p2(22/38), p4(26/19),p5(8/41),p6(45/30)	50
Modul 4: Neuronale Netzwerke.....	50
1. Skizzieren Sie ein neuronales Netz und benennen Sie dessen Elemente.....	50
2. Wie funktioniert das Trainieren eines neuronalen Netzes mittels Backpropagation? (Es sind keine Formeln notwendig, die Erklärung der Idee reicht)	50
3. Was sind Deep Neural Networks?	50
4. Was sind Convolutional Neural Networks (CNNs)?.....	50
5. Erklären Sie die Grundidee der Funktionsweise von ChatGPT und ähnlichen Systemen.....	51
Clingo Aufgaben	51
Schengen Countries 1	51
Schengen Countries 2	51
Birds.....	52
Cousin.....	52
Adjacent States	53
Public Transport 1	53
Public Transport 2	54
Father/Mother.....	54
Missouri.....	55
Nodes	55

Einführung in AI (Folien)

Modul 1: Grundlagen, Ausrichtungen, Anwendungen

1. Definitionen und Überblick über KI-Ausrichtungen

Definition

Künstliche Intelligenz (KI)

Es ist schwer, eine exakte Definition festzulegen. McCarthy (1955): „Ziel der KI ist es, Maschinen zu entwickeln, die sich verhalten, als verfügten sie über Intelligenz“.

Maschinelles Lernen (ML)

Die besondere Stärke menschlicher Intelligenz ist die **Adaptivität** (Fähigkeit, sich durch Lernen an verschiedenste Umweltbedingungen anzupassen).

ML ist die Erforschung „intelligenter“ Verfahren, die die Lernfähigkeit zu imitieren versuchen. Ebenfalls ist es der Versuch, das menschliche Gehirn zu modellieren und verstehen.

AI besteht nicht *nur* aus ML, ML ist nicht *nur* deep learning! (**KI $\supset$ ML $\supset$ Neuronale Netze $\supset$ Deep Learning**).

Die KI-Apotheke

Es gibt in der KI keine universelle Methode für alle Anwendungsbereiche. Viele Verfahren stammen aus der Kognitionswissenschaft (Hirnforschung), z.B. dem *menschlichen Schlussfolgern* (Logik, Schließen mit Unsicherheit), das *Gehirn* selbst (Neuronale Netze) oder *Such- und Problemlösungsstrategien*.

Auswahl einer KI-Methode

Ziel ist die Entwicklung eines intelligenten Agenten:

1. Formulierung des Problems
2. Finden der optimalen Lösungsstrategie
3. Wahl der Theorie (z.B. Prädikatenlogik) und Methode

Beispiel: Medizin – Diagnoseunterstützende Software

- Schlussfolgerung anhand von Symptomen
- Z.B. durch Logik-basierte Agenten



Turing-Test und Chatbots

Alan Turing: KI-Pionier durch die Definition einer intelligenten Maschine.

Turing Test: Eine Testperson hat Zugang zu zwei Terminals zur Kommunikation – einmal mit einem Menschen und einmal mit einer Maschine. „Eine Maschine kann als intelligent bezeichnet werden, wenn sie die Testperson in 30% der Fälle täuschen kann.“

Diese Definition ist zwar philosophisch interessant, aber in der Praxis nicht relevant. Es war allerdings die Basis für den ersten Chatbot („Eliza“ von Joseph Weizenbaum).

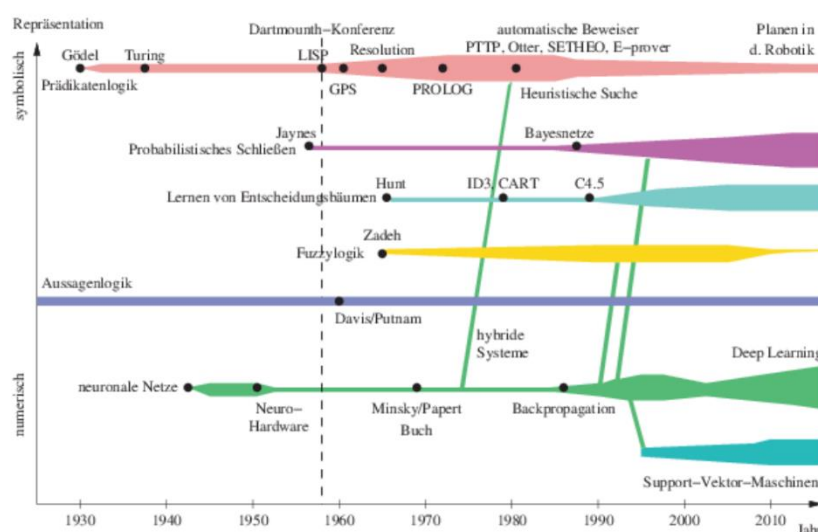
Chatbots sind für bestimmte Anwendungen schon länger eine verbreitete Technologie, seit kurzem auch für „allgemeine Intelligenz“ (z.B. ChatGPT).

2. Geschichtliche Entwicklung

Ausgewählte Meilensteine der KI

Jahr	Meilenstein
1931	Kurt Gödel (AT) beweist Vollständigkeit der Prädikatenlogik erster Ordnung
1937	Alan Turing zeigt mit Halteproblemen die Grenzen von KI auf
1943	Erstes Modell eines neuronalen Netzes (McCullocks/Pitts)
1950	Turing-Test, erste Arbeit zu lernenden Maschinen und genetischen Algorithmen
1951	Erster Neuronenrechner (3000 Röhren simulieren 40 Neuronen)
1956	Erste AI-Konferenz
1958	LISP (McCarthy) – Programme, die sich selbst verändern können
1966	Eliza führt Dialoge mit Menschen in natürlicher Sprache (Weizenbaum)
1972	Logikprogrammiersprache Prolog (Colmerauer)
1976	MYCIN (Shortcliff/Buchanan), ein logikbasiertes medizinisches Diagnosetool
1990	Bayes-Netze (Wahrscheinlichkeitstheorie in der KI)
1993	Start des RoboCup
1997	Deep Blue (IBM) besiegt Gary Kasparov
2009	Erste Fahrt eines selbstgesteuerten Fahrzeugs (Google)
2011	Watson (IBM) schlägt zwei Jeopardy Meister
2015	Deep Learning verbessert Klassifikation von Bildern. Zudem Generierung von Gemälden im Stil alter Meister (ebenfalls DL).
2016	AlphaGo (Google DeepMind) schlägt weltbeste GO-Spieler (Deep Learning, Reinforcement Learning und Monte Carlo Tree Search).
2022	Der Chatbot ChatGPT von OpenAI überrascht auch die Fachwelt und geht in Richtung Strong AI (Artificial General Intelligence).
2023-	Seither entstanden weitere ähnliche Systeme anderer Entwickler

Überblick über die KI Entwicklung



Aktueller Trend in den 2020er Jahren: Large Language Models (LLMs) – basieren selbst auf Deep Learning.

Entwicklung und Trends

Die ersten Ansätze waren symbolisch und stark logikbasiert.

Heute:

- Trend 1: Immer mehr Ansätze basieren auf Daten. Big Data als klarer Innovator, aber auch als Herausforderung
- Trend 2: Das Verknüpfen von unterschiedlichen KI-Ausrichtungen erzielte jüngst beachtliche Erfolge (Deep Learning)
- Trend 3: Auch alte Technologien (regelbasierte Ansätze wie Chatbots) verbreiten sich immer weiter. Viel Potenzial durch Experimente in neuen Bereichen.

3. Gesellschaft

Gesellschaftliche Aspekte

Es gibt sehr viel Science Fiction, oft mit beängstigenden Darstellungen von KI. Verlust von Arbeitsplätzen ist ebenfalls eine Sorge vieler Menschen (z.B. selbstfahrende PKWs oder Chatbots). Viele dieser Jobs sind allerdings schwer, schmutzig und/oder ungesund. Monotone Tätigkeiten können von Maschinen auch schneller, exakter und kostengünstiger erledigt werden (z.B. First Level Support). Es gehen also oft unangenehme Arbeiten verloren -> Menschen müssen sich allerdings für höherwertige Stellen besser qualifizieren. Notwendigkeit dieser Entwicklung entsteht auch durch Konkurrenzdruck (z.B. billige Produktion in China).

KI und Verkehr

Seit Jahrzehnten werden Fahrzeuge für Komfort und Sicherheit immer mehr computerisiert. Die nächste bevorstehende Stufe sind **selbstfahrende Autos**:

- Keine kognitive Belastung beim Fahren
- Weniger Fahrzeuge notwendig (mehr Sharing durch selbstfahrende Taxis)
- Optimale Nutzung von KFZs durch Computer, also weniger Energieverbrauch, Verschleiß und Schadstoffe
- Neue Probleme: Rechtliches (Unfälle), Rebound Effekt (mehr Komfort -> mehr Nutzung), Vernetzung und Datensammeln (Hackerangriffe, Datenlecks, ...)

Servicerobotik

Roboter benötigen das Zusammenspiel von Hardware und Software (-> Software ist für die „Intelligenz“ der Maschine verantwortlich). Es gibt verschiedene Anwendungen wie z.B. Serviceroboter (Komfort im Haushalt), „Intelligente“ Prothesen in der Medizin oder Jobs, die für Menschen zu gefährlich sind.

4. Technologien

Ausrichtungen von KI

Logik-basierte KI

Ausführen von Computerprogrammen auf Basis mathematischer Logik. Es gibt *Fakten und Regeln* zum Lösen von Problemen. Beispiele: Mustererkennung, Sprachverarbeitung, semantische Analyse.

Anwendungsbereiche:

- Regelbasierte Expertensysteme wie MYCIN (unterstützendes medizinisches Diagnosetool)
- Grundlage für andere Verfahren wie ML
- Regelbasiertes Schlussfolgern, regelbasiertes Modellieren
- Rewriting - Programme die sich selbst abwandeln können

Prädikatenlogik 1. Stufe (First Order Logic, FOL)

■ Beispiel: Verwandtschaftsverhältnisse

$$(\forall X) (\text{parent}(X) \leftrightarrow (\text{human}(X) \wedge (\exists Y) \text{parent_of}(X,Y)))$$

$$(\forall X) (\text{human}(X) \rightarrow (\exists Y) \text{parent_of}(Y,X))$$

$$(\forall X) (\text{orphan}(X) \leftrightarrow (\text{human}(X) \wedge \neg (\exists Y) (\text{parent_of}(Y,X) \wedge \text{alive}(Y))))$$

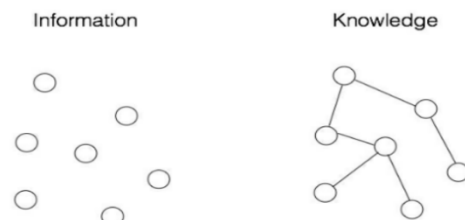
$$(\forall X)(\forall Y)(\forall Z) (\text{uncle_of}(X,Z) \leftrightarrow (\text{brother_of}(X,Y) \wedge \text{parent_of}(Y,Z)))$$

Intendierte Semantik: klar!

Web, Dr. Harald Sack, Heisen-Partner-Institut, Universität Potsdam

Wissensrepräsentation

(Menschenverständliche) Darstellung von Fakten und Zusammenhängen. Oftmals ist Logik die Basis solcher Darstellungen. Das Ziel ist es, Wissen effizient darzustellen. Anwendungsbereiche: Ontologien, Taxonomien.

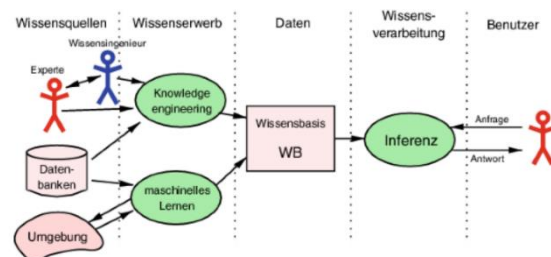


Technologie – Wissensbasierte Systeme

Komplexe KI-Anwendungen, für die eine große Menge an Wissen notwendig sein kann (z.B. umfangreiches Regelwerk, großer Faktenpool).

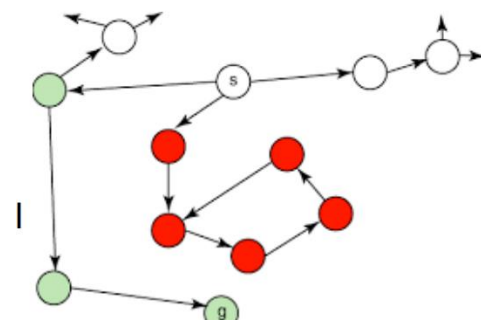
Strukturierter Ansatz:

1. Erwerb des Wissens durch Knowledge Engineering oder durch lernende Systeme
2. Speicherung in einer Wissensbasis
3. Wissen durch KI-Anwendung nutzen -> Inferenzmechanismus



Suche

Suchtechniken die für KI notwendig sind. Problembereich: große Anzahl an Möglichkeiten, etwa Netzwerke mit vielen Knoten und Übergängen -> viele mögliche Pfade. Deshalb braucht es Strategien, um den optimalen Pfad zu finden (wobei „optimal“ per Zielsetzung definiert ist). Anwendungen hierfür gibt es z.B. bei Strategiespielen (Schach, Go) oder Ressourcenallokation/Scheduling in Computernetzwerken.



Planung

Optimale Planung = maximaler Ertrag bei minimalem Aufwand.

Um den optimalen Plan zur Zielerreichung zu finden, benötigt es Fakten über die aktuelle Situation und die Zieldefinition. Es gibt dann ein internes (etwa fakten- und regelbasiertes) Modell der Welt. Anwendungsbereiche: Intelligente Agenten, Planung unter Berücksichtigung von Constraints.

Heuristik

Technik, um ein bestehendes Problem schnellstmöglich zu lösen, wobei die optimale Lösung nicht garantiert sein muss (ähnlich wie ein „Educated Guess“) -> unter Nutzung von Heuristiken (einfache Regeln, z.B. basierend auf Menschenverstand oder Erfahrungen). Dieser Ansatz ist oft notwendig, wenn der Lösungsraum zu groß ist. Anwendungsbeispiele: Schließen mit Unsicherheit, Robotik, Suchmaschinen.

Evolutionäre Programmierung

Problemlösung durch **Vergleichen und Kombinieren unterschiedlicher**

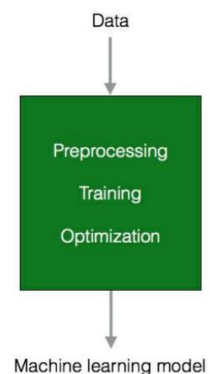
Verfahren/Algorithmen -> Auswahl des besten Algorithmus („**survival of the fittest**“), Ensemble Learning. Hier gibt es eine „Fitnessfunktion“ welche die Erfolgsquote der Programme, welche die vorgegebene Aufgabe am besten erledigen, berechnet. „Erfolg“ ist hier durch verschiedene Faktoren definiert („Effektivität, Effizienz, Performance, ...“).

Dieses Verfahren kann grundsätzlich immer verwendet werden, allerdings ist es auch ein hoher Aufwand im Vergleich zu anderen Auswahlverfahren (Nutzerwertanalyse, ...).

Maschinelles Lernen & Data Mining

Derzeit die spannendste Form der KI-Ausprägungen (aufgrund vorhandener Daten in vielen Bereichen) -> „**datenbasierte AI**“. Ein Softwareprogramm lernt aus einer Basis von Daten -> das „erlernte“ Modell kann dann verwendet werden, um Vorhersagen über unbekannte Daten (z.B. auftretende Ereignisse) zu erstellen. Eine Voraussetzung dafür ist ein Vorhandensein „guter“ (=> Größe, Qualität) Datensätze.

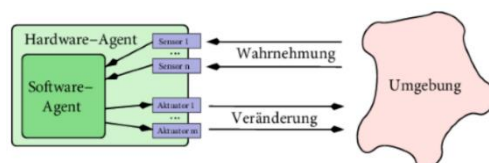
Anwendungsbereiche: Predictive Analytics, Mustererkennung (Merkmalsextraktion) -> Gesichtserkennung, Spracherkennung.



Technologie – Agenten

Ein „Agent“ ist ein System, welches Informationen verarbeitet und aus einer Eingabe eine Ausgabe produziert.

- Software-Agent (Informatik): Ein Programm errechnet aus einer Benutzereingabe ein Ergebnis
- Hardware-Agent (Robotik): Agenten mit eigenen Sensoren und Aktuatoren -> Wahrnehmung und Veränderung der Umgebung



Agenten benötigen eine **Zielorientierung** -> das Ziel bestimmt die Aktionen. Dafür ist die *Wahrheitsmatrix* (*Confusion Matrix*) ein wichtiges Qualitätskriterium.

Siehe Beispiel: Agent 1 macht weniger Fehler (12), aber 11 davon sind schwerwiegende (11 erwünschte Mails als Spam gefiltert).

Diagonale: true-true bzw. false-false Werte

(Agent handelt korrekt). Abseits der Diagonale: true-false bzw. false-true (Fehler).

Agent 1:				Agent 2:			
Spam-Filter entscheidet		korrekte Klasse		Spam-Filter entscheidet		korrekte Klasse	
		erwünscht	Spam			erwünscht	Spam
erwünscht	erwünscht	189	1	erwünscht	erwünscht	200	38
	Spam	11	799		Spam	0	762

Kostenorientierung: unterschiedliche Gewichtung der Fehler.

5. Anwendungsbereiche

Maschinelles Sehen (Bildanalyse)

Systeme, die Bilddaten und Videos verarbeiten und unter anderem versuchen, die Fähigkeiten der menschlichen visuellen Wahrnehmung zu imitieren. Typischerweise wird hier das Verfahren des maschinellen Lernens und des Data Minings verwendet. Anwendungsfälle hierfür sind: Mustererkennung und Merkmalsextraktion (Gesichtserkennung, alle Formen der Objekterkennung in visuellen Daten), auch Bildersuche.

Schwierigkeiten:

- Unterdrückung vielschichtiger Umgebungsbedingungen
- Erkennung (Suche) von abstrakten Konzepten wie Emotionen, Lügen, ... in visuellen Daten

Linguistische Datenverarbeitung

Natürliche Sprache in Form von Text- oder Sprachdaten mit Hilfe des Computers verarbeiten.

Anwendungsbereiche: automatische Korrektur, Text Mining, Übersetzung von Texten, Generierung von natürlichsprachigen Texten, Zusammenfassungen, ...

Schwierigkeiten:

- Auflösung syntaktischer Mehrdeutigkeiten („Wir essen jetzt, Oma“ vs. „Wir essen jetzt Oma“)
- Bestimmen der Semantik („Ich habe einen Virus“)
- Erkennen der Absicht einer sprachlichen Äußerung (Pragmatik)
 - Z.B. Sarkasmus

Spracherkennung

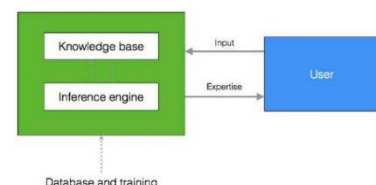
Systeme, die in der Lage sind, gesprochene Wörter aufzuzeichnen und zu erkennen.

Beispiel: intelligenter persönlicher Assistent (Smartphone, Echo, ...) -> Sprachinput führt zu unmittelbaren Aktionen (relevante Informationen liefern, Steuersignale senden, ...)

Dies ist eine Kombination aus unterschiedlichsten Verfahren auf Hard- und Softwareebene (Digitalisierung des Audiosignals, Mustererkennung, linguistische Datenverarbeitung, ...).

Expertensysteme

Systeme, die KI-Techniken nutzen, um Ratschläge/Feedback zu geben oder sogar Entscheidungen zu treffen. Hierfür benötigt man eine Wissensbasis und eine Inferenzmaschine. Durch „Knowledge Engineers“ wird die Wissensbasis aufgebaut.



Spiele

Seit jeher wird AI in der Spieleindustrie für intelligente Gegner oder intelligentes Softwareverhalten verwendet. Auch hier ist es möglich unterschiedliche KI-Techniken zu verwenden, der Mensch ist allerdings bei einzelnen Verfahren überlegen.

Ein kombinierter Einsatz solcher intelligenten Strategien (Deep Learning) -> IBM Deep Blue besiegt Schachweltmeister, AlphaGo besiegt zwei Go-Weltklassemaster.

Robotik

Ebenfalls ist es hier eine Kombination unterschiedlicher Strategien sowie Hardware (Elektronik + Mechanik) und Software (Steuerung + Gedächtnis). Abhängig von den zu erreichenden Aufgaben gibt es unterschiedliche Sensoren (Bewegung, Temperatur, Helligkeit, ...) und Aktuatoren in allen möglichen Ausrichtungen.

Eckpunkte des Moduls

Künstliche Intelligenz steht für den Versuch, menschliche Intelligenz mit einem Computersystem (Hard- und Software) nachzuahmen.

Maschinelles Lernen (mit/ohne Lehrer, mit Verstärkung) meint Verfahren, die die Lernfähigkeit des Menschen zu imitieren versuchen, und ist derzeit aufgrund vorhandener Echtzeiten stark im Fokus (vgl. Data Scientist). Das ist nur ein Teil der KI!

Neben **traditionellen Ausrichtungen der KI** (Logik, Suche, Wissensrepräsentation, Schließen) wird derzeit vor allem an **datenbasierten Ansätzen** (Data Mining, maschinelles Lernen) sowie an **Praxisnähe** von bestehenden Verfahren (Schließen mit Unsicherheit, Heuristik, evolutionäre Programmierung) und der **Kombination von unterschiedlichen Ansätzen** (Deep Learning, Ensemble Learning) gearbeitet.

Beispielhafte KI-Technologien sind **Agenten** wie auch **wissensbasierte Systeme** zu nennen.

Anwendungsbereiche sind u.a. die Bildanalyse, die Spracherkennung und linguistische Datenverarbeitung, Expertensysteme, Spiele oder die Robotik.

Modul 2: Logik

1. Aussagenlogik

Was versteht man unter Aussagenlogik?

Eine mathematische Sprache, um Wissen in strukturierter und maschinenlesbarer Form darzustellen. Aussagen werden über logische Operationen verknüpft (siehe Abbildung).

wenn es regnet ist die Straße nass.

$es\ regnet \Rightarrow die\ Straße\ ist\ nass.$

Vorteil der 2. Schreibweise: maschinenlesbarer und die Struktur ist unabhängig vom Inhalt von Basisaussagen.

Syntax

Definition 2.1

Sei $Op = \{\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow, (,)\}$ die Menge der logischen Operatoren und Σ eine Menge von Symbolen. Die Mengen Op, Σ und $\{w, f\}$ seien paarweise disjunkt. Σ heißt **Signatur** und seine Elemente sind die **Aussagevariablen**. Die Menge der Aussagenlogischen Formeln wird nun rekursiv definiert:

- w und f sind (atomare) Formeln.
- Alle Aussagevariablen, das heißt alle Elemente von Σ sind (atomare) Formeln.
- Sind A und B Formeln, so sind auch $\neg A$, (A) , $A \wedge B$, $A \vee B$, $A \Rightarrow B$, $A \Leftrightarrow B$ Formeln.

Definition 2.2

Man liest die Operatoren und Symbole folgendermaßen:

w : „wahr“

f : „falsch“

$\neg A$: „nicht A “ (Negation)

$A \wedge B$: „ A und B “ (Konjunktion)

$A \vee B$: „ A oder B “ (Disjunktion)

$A \Rightarrow B$: „wenn A dann B “
(Implikation ¹)

$A \Leftrightarrow B$: „ A genau dann, wenn B “
(Äquivalenz)

Zudem gilt (Abschlussbedingung): Formeln werden nur über diese Regeln gebildet, d.h. Ausdrücke, die nicht über diese rekursive Definition gebildet werden können, sind nicht zulässig (etwa „ $A B$ “).

Möglichkeiten dieser Syntax

Durch Definition sind unendlich viele Formeln möglich.

Beispiel: gegeben $\Sigma = \{A, B, C\}$. Syntaktisch korrekt: $((A) \vee B)$. Aber auch

$A \wedge B$, $A \wedge B \wedge C$, $A \wedge A \wedge A$, $C \wedge B \vee A$, $(\neg A \wedge B) \Rightarrow (\neg C \vee A)$

Allerdings ist das nur das syntaktische Gebilde. Es fehlt noch die Semantik (=Bedeutung)!

Semantik

Zwei Wahrheitswerte: wahr und falsch. Ist die Formel $A \wedge B$ wahr? -> hängt von den Werten von A und B ab. Beispiel: A = „es regnet“, B = „es ist kalt“. Ob „regnet UND kalt“ gilt, hängt von den Werten von A bzw. B ab.

Aussagenvariablen müssen Wahrheitswerte zugeordnet werden, die Zuständen der Welt entsprechen.

Definition 2.3

Eine Abbildung $I: \Sigma \rightarrow \{w, f\}$, die jeder Aussagevariablen einen Wahrheitswert zuordnet, heißt **Belegung** oder **Interpretation** oder auch **Welt**.

Jede aussagenlogische Formel mit n Variablen besitzt 2^n verschiedene Belegungen (bzw. Welten).

Eine Wahrheitstabelle definiert resultierende Werte für die erlaubten Basisoperatoren.

Wahrheitstabelle für logische Operatoren

Über 2 Variablen (A, B) sind $2^2 = 4$ Interpretationen möglich

A	B	(A)	$\neg A$	$A \wedge B$	$A \vee B$	$A \Rightarrow B$	$A \Leftrightarrow B$
w	w	w	f	w	w	w	w
w	f	w	f	f	w	f	f
f	w	f	w	f	w	w	f
f	f	f	w	f	f	w	w

Der Wert einer atomaren Formel (= eine alleinstehende Variable) wird direkt durch die Interpretation bestimmt. Zur Ermittlung der Wahrheitswerte für komplexe Formeln ist die Rangfolge der logischen Operatoren notwendig -> zuerst die Klammerung, dann die folgende Reihenfolge: $\neg$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$

Äquivalenz – Syntax vs. Semantik

Definition 2.4

Zwei Formeln F und G heißen semantisch äquivalent, wenn sie für alle Belegungen die gleichen Wahrheitswerte annehmen. Man schreibt $F \equiv G$.

Im Unterschied zur syntaktischen Äquivalenz. Semantik = Metasprache (nämlich die natürliche Sprache) über die Objektsprache (die Logik).

Definition 2.5

Eine Formel heißt

- **erfüllbar**, falls sie bei mindestens einer Belegung wahr ist.
- **allgemeingültig** oder **wahr**, falls sie bei allen Belegungen wahr ist. Wahre Formeln heißen auch **Tautologien**.
- **unerfüllbar**, falls sie bei keiner Belegung wahr ist.

Jede erfüllende Belegung einer Formel heißt **Modell** der Formel.

Satz 1

Die Operationen $\wedge, \vee$ sind kommutativ und assoziativ und folgende Äquivalenzen sind allgemeingültig:

$\neg A \vee B \Leftrightarrow A \Rightarrow B$	(Implikation)
$A \Rightarrow B \Leftrightarrow \neg B \Rightarrow \neg A$	(Kontraposition)
$(A \Rightarrow B) \wedge (B \Rightarrow A) \Leftrightarrow (A \Leftrightarrow B)$	(Äquivalenz)
$\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$	(De Morgan'sche Regeln)
$\neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B$	
$A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$	(Distributivgesetze)
$A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$	
$A \vee \neg A \Leftrightarrow w$	(Tautologie)
$A \wedge \neg A \Leftrightarrow f$	(Widerspruch)
$A \vee f \Leftrightarrow A$	
$A \vee w \Leftrightarrow w$	
$A \wedge f \Leftrightarrow f$	
$A \wedge w \Leftrightarrow A$	

Beweis ,Implikation':

A	B	$\neg A$	$\neg A \vee B$	$A \Rightarrow B$	$(\neg A \vee B) \Leftrightarrow (A \Rightarrow B)$
w	w	f	w	w	w
w	f	f	f	f	w
f	w	w	w	w	w
f	f	w	w	w	w

Die Folgerung (Schließen, Inferenz)

Ziel von logikbasierten Ansätzen: aus bestehendem Wissen neues Wissen abzuleiten. D.h aus Wissensbasis WB (ev. umfangreiche aussagenlogische Formel) folgt eine Formel Q. In jeder Welt, in der WB wahr ist, ist auch Q wahr, kurz: Immer, wenn WB wahr ist, ist auch Q wahr.

Definition 2.6

Eine Formel Q **folgt** aus einer Formel WB, wenn jedes Modell von WB auch ein Modell von Q ist. Man schreibt dann $WB \models Q$.

Die Folgerung $WB \models Q$ ist eine semantische (metasprachliche) Aussage über zwei Formeln WB und Q: „Jedes Modell von WB ist ein Modell von Q“. Dagegen ist $WB \Rightarrow Q$ eine Formel (die Implikation $\Rightarrow$ ist ein logischer Operator als Teil der Formel).

Beispiel: $A \wedge B \models A \vee B$
aber $A \vee B \not\models A \wedge B$

Berechenbarkeit / Komplexität

Komplexität: Check auf Erfüllbarkeit von aussagenlogischen Formeln i.A. NP-vollständig (worst case), praktische Anwendungen fallen aber zum Glück meistens nicht in den worst case.

Wahrheitstafeln: Nicht dabei hängen bleiben, bloß weil einfach. Werden zwar verwendet, aber nicht glauben man kommt damit allein aus:

- Logik ist viel mehr, zentral ist z.B. die **Folgerung**
- In der Praxis sind Wahrheitstafeln auch viel zu ineffizient, Logikprogrammiersysteme verwenden sie daher **nicht**
- Auch für erweiterte Logik (z.B. Prädikatenlogik) **nicht** anwendbar

Anwendungen

- Aussagenlogische Beweiser in der Digitaltechnik (Verifikation digitaler Schaltungen)
- Binäre Entscheidungsbäume in der Mustererkennung
- Logikbasierte Expertensysteme

2. Prädikatenlogik

Prädikatenlogik: Mächtiger Lösungsansatz

Ermöglicht Aussagen abhängig von Parametern, etwa

$$\forall u \forall v \text{ Steht_weiter_rechts}(u, v) \Leftrightarrow$$

$$\exists x_u \exists y_u \exists x_v \exists y_v \text{ Position}(u, x_u, y_u) \wedge \text{Position}(v, x_v, y_v) \wedge x_u > x_v$$

(Notation: „für alle u“, „es existiert ein x_u“)

Motivation

Aussagenlogik ist zwar in vielen Bereichen ausreichend, aber für größere Anwendungen ist Prädikatenlogik nützlicher und einfacher.

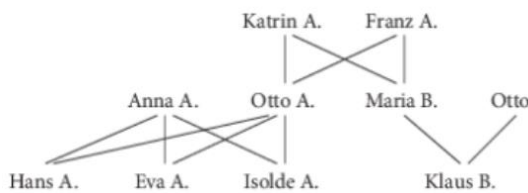
(z.B. Gitter mit 100x100 Positionen auf der sich 10 Roboter befinden können => mit Aussagenlogik 100x100x10 = 100000 Variablen).

*„Roboter_7_befindet_sich_an
xy-Position(35,79)“*

Beispiele

Formel	Beschreibung
$\forall x \text{ frosch}(x) \Rightarrow \text{grün}(x)$	Alle Frösche sind grün
$\forall x \text{ frosch}(x) \wedge \text{braun}(x) \Rightarrow \text{groß}(x)$	Alle braunen Frösche sind groß
$\forall x \text{ mag}(x, \text{kuchen})$	Jeder mag Kuchen
$\neg \forall x \text{ mag}(x, \text{kuchen})$	Nicht jeder mag Kuchen
$\neg \exists x \text{ mag}(x, \text{kuchen})$	Keiner mag Kuchen
$\exists x \forall y \text{ mag}(y, x)$	Es gibt etwas, das jeder mag
$\exists x \forall y \text{ mag}(x, y)$	Es gibt jemanden, der alles mag
$\forall x \exists y \text{ mag}(y, x)$	Jedes Ding wird von jemandem geliebt
$\forall x \exists y \text{ mag}(x, y)$	Jeder mag etwas
$\forall x \text{ kunde}(x) \Rightarrow \text{mag}(\text{bob}, x)$	Bob mag jeden Kunden
$\exists x \text{ kunde}(x) \wedge \text{mag}(x, \text{bob})$	Es gibt einen Kunden, der Bob mag
$\exists x \text{ bäcker}(x) \wedge \forall y \text{ kunde}(y) \Rightarrow \text{mag}(x, y)$	Es gibt einen Bäcker, der all seine Kunden mag
$\forall x \text{ älter}(\text{mutter}(x), x)$	Jede Mutter ist älter als ihr Kind
$\forall x \text{ älter}(\text{mutter}(\text{mutter}(x)), x)$	Jede Großmutter ist älter als das Kind ihrer Tochter
$\forall x \forall y \forall z \text{ rel}(x, y) \wedge \text{rel}(y, z) \Rightarrow \text{rel}(x, z)$	rel ist eine transitive Relation

Beispiel: Stammbaum



Rekursive Definition von „Nachkomme“:

$$\forall x \forall y \text{ nachkomme}(x, y) \Leftrightarrow \exists z \text{ kind}(x, y, z) \vee (\exists u \exists v \text{ kind}(x, u, v) \wedge \text{nachkomme}(u, y)).$$

Anwendungen

- Theorembeweiser in der Mathematik
- Expertensysteme
- Automatische Programmverifikation
- Semantic Web

3. Logikorientierte Programmierung

Durch ein Logikprogramm wird Wissen beschrieben. Der **Interpreter** (oder **Reasoner, Inferenz-Engine**) des Logikprogramms schließt dann darauf, was aus diesem Wissen folgt.

Allerdings sollte man keine falschen Erwartungen haben:

- Logikorientierte Sprachen sind in der Regel nicht als Universalsprache konzipiert. Sie wollen keine Alternative zu OOP (C, C++, Java, etc.) sein, sondern eine Ergänzung (z.B. als Backend)
- Bekannte Konstrukte wie If-Verzweigungen, Schleifen, Prozeduren, ... gibt es nicht. Auch spielt bei vielen Semantiken nicht einmal die Reihenfolge der Regeln eine Rolle (es gibt keine Abarbeitung von oben nach unten, sondern das Programm muss ganzheitlich erfüllt sein)

Logikorientierte Sprachen sind daher eher mit Abfragesprachen wie SQL zu vergleichen als mit prozeduraler Programmierung/OOP.

Beschreibung des Wissens verwendet spezielle Formeln der PL1 der folgenden Form, sogenannte **Regeln**: $B^I \vee \dots \vee B^W \vee \neg B^{W+I} \vee \dots \vee \neg B^U \Rightarrow H^I \wedge \dots \wedge H^K$

Informell: Wenn alle B 's (der *Regelkrumpf*) wahr sind, dann muss auch zumindest eines der H 's (der *Regelkopf*) wahr sein.

Syntax in der Praxis (ASCII-basiert):

$H_1 \vee \dots \vee H_k :- B_1, \dots, B_m, \text{not } B_{m+1}, \dots, \text{not } B_n$
(oder systemabhängig)

$H_1 \mid \dots \mid H_k :- B_1, \dots, B_m, \text{not } B_{m+1}, \dots, \text{not } B_n$

Wobei die B 's und H 's Atome der Form $P(x_1, \dots, x_l)$ sind.

Konventionen

Namenskonvention in den meisten Systemen:

- Prädikate p sind klein geschriebene Strings
- Argumente x sind entweder
 - *Konstanten* (klein geschrieben oder in “ eingeschlossen)
 - *Variablen* (großgeschrieben)

Konvention: wenn $k=0$ („leerer“ Regelkopf) $\Rightarrow$ false. Die gesamte Regel steht daher für $B_1 \wedge \dots \wedge B_m \wedge \neg B_{m+1} \wedge \dots \wedge \neg B_n \Rightarrow \perp$ und wird **Constraint** genannt.

Bedeutung: der Regelrumpf darf nie wahr werden (sonst wäre die Implikation nicht erfüllt).

Terminologie

Bestimmte Regeltypen haben Sondernamen:

- **Fakt** wenn $n=0$ und $k=1$. Beispiel: *employee(sue)*. (etwas ist wahr ohne Vorbedingung)
- **Disjunkter Fakt** wenn $n=0$ und $k>0$. Beispiel: *cloudy | sunny | rainy*. (eine von mehreren Optionen ist wahr ohne Vorbedingung)
- **Constraint** wenn $k=0$. Beispiel: *:-married(X), single(X)*. (der leere Regelkopf steht implizit für false $\rightarrow$ drückt aus, dass der Rumpf niemals wahr werden darf)

Komponenten

Ein Logikprogramm besteht aus zwei Komponenten:

- **Intensionale Datenbank (IDB): Allgemeine** Problembeschreibung
- **Extensionale Datenbank (EDB): Konkrete Instanz** durch Fakten

Beispiel: Berechnung des transitiven Abschlusses der Kanten in einem Graphen

IDB={ *edge(Y, Z) :- edge(X, Y), edge(Y, Z) .* }

EDB={ *edge(a, b) . edge(c, d), edge(u, v) .* }

Beachte: Die Trennung hat rein praktischen Nutzen (z.B. Aufteilung in zwei Dateien). Formal bilden beide zusammen **eine** Menge von Regeln!

Intuition

Regeln drücken von rechts nach links gelesen eine Implikation aus. Z.B.

edge(Y, Z) :- edge(X, Y), edge(Y, Z)

„Wenn (X,Y) eine edge ist, und (Y,Z) ebenso, dann ist auch (Y,Z) eine edge“

Beachte:

- *edge* ist **kein** Schlüsselwort, sondern ein Prädikat! Es könnte genauso gut *kante(X,Y)* oder *blafoo(X,Y)* heißen
- Tatsächlich gibt es hier keine Schlüsselwörter. **Kein** Prädikat hat irgendeine inhärente Bedeutung in der Sprache, sinnvolle Bezeichnungen dienen nur der Verständlichkeit durch die User

- Trotz Formulierung „Wenn...dann“ ist eine Implikation was anderes als eine If-Verzweigung in der prozeduralen Programmierung! Insbesondere wird dadurch auch kein zeitlicher Ablauf definiert -> eine Lösung muss die Regeln insgesamt erfüllen.

Beispiele

Beispiel: Reachability

IDB:

```
reach(X, Y) :- conn(X, Y) .
conn(X, Z) :- conn(X, Y), conn(Y, Z)
```

EDB: Menge von Fakten über Prädikat *conn* z.B.

```
conn(heiligenstadt, spittelau) .
conn(spittelau, friedensbrücke)
```

Beispiel: Familienstammbaum (nur IDB)

```
mother(C, M) :- childOf(M, C), female(M) .
father(C, F) :- childOf(F, C), male(F) .
sibling(C, S) :- childOf(X, C), childOf(X, S) .
cousin(C1, C2) :- childOf(C1, P1), childOf(C2, P2),
                  sibling(P1, P2) .
```

Logikprogrammiersysteme

Semantik:

- Es gibt in der Praxis nicht **die** logikorientierte Programmierung, sondern verschiedene Semantiken
- Für jede Semantik gibt es mitunter mehrere Implementierungen, die teilweise leichte Unterschiede vorweisen oder proprietäre Erweiterungen anbieten

Wir verwenden die **Answer Set-Semantik (Antwortmengenprogrammierung)** als Beispiel und deren Implementierung im System **clingo**. Alles bisher Gesagte gilt unter anderem auch dafür.

Alternativen (z.B. Prolog, kanren für Python, ...) unterscheiden sich im Detail mehr oder weniger stark, basieren aber auf ähnlichen Grundideen.

Logikprogrammierung nach der Answer Set Semantik

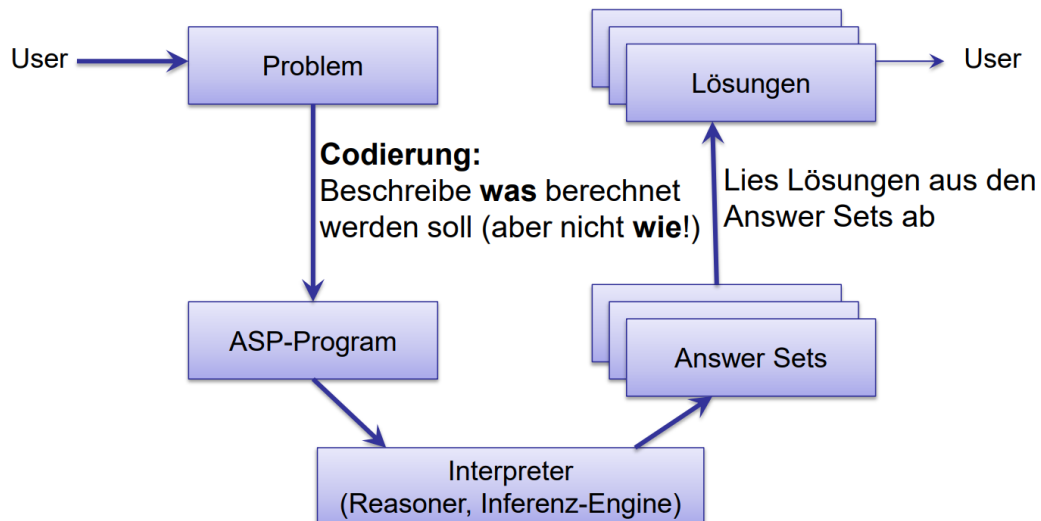
Idee:

- Ein **Programm** ist eine Menge von Regeln obiger Art
- Es hat 0 bis mehrere (endlich viele) Lösungen in Form von sogenannten **Answer Sets** (=Antwortmengen)
- Ein **Answer Set** ist eine Menge von wahren Atomen (vgl. Interpretationen in Aussagenlogik), die
 - alle Regeln erfüllt und
 - minimal ist (Minimalität stellt sicher, dass nichts grundlos auf wahr gesetzt wird)

Beispiel für Minimalität:

```
single :- not married.  
married :- not single.
```

Obwohl $\{single\}$, $\{married\}$ und $\{single, married\}$ jeweils beide Regeln erfüllen, sind nur die ersten beiden Lösungen relevant, weil die letzte nicht minimal ist.



System clingo

clingo ist Open-Source und das derzeit am weitesten verbreitete System unter der Answer Set Semantik.

Manual: <https://github.com/potassco/guide/releases/download/v2.2.0/guide.pdf>

Video-Tutorials: <https://www.youtube.com/c/Potassco-live/videos>

Aufruf (command line): `clingo -n 0 program.asp`. Das Programm kann zur Strukturierung auch in mehrere Dateien geteilt werden (es sind jedoch alle Regeln gleichwertig und verhalten sich als wären sie in einer Datei).

Wichtig: clingo gibt standardmäßig nur die **erste** gefundene Lösung aus! Option `-n <N>` erlaubt die Berechnung von `<N>` Lösungen.

Eckpunkte des Moduls

- **Aussagenlogik** als erster (ältester) Formalismus, um eine Wissensdatenbank aufzubauen und logische Schlussfolgerungen darauf aufzubauen.
- **Syntaktische Korrektheit** sowie **semantische Korrektheit und Vollständigkeit** gefordert, allerdings in vielen Fällen nicht berechenbar (vgl. Compiler)
- **Prädikatenlogik** mächtiger als Aussagenlogik
- **Logikorientierte Programmierung** am Beispiel einer konkreten Semantik und des Systems clingo

Modul 3: Maschinelles Lernen

1. Überblick Machine Learning

Was ist maschinelles Lernen?

- Teilgebiet der Künstlichen Intelligenz
- Erforschung „intelligenter“ Verfahren, die die menschliche Lernfähigkeit zu imitieren versuchen
- Auch der Versuch, die Funktionsweise des menschlichen Gehirns zu modellieren und verstehen
- Unterschiedliche Formalismen und Vorgehensweisen

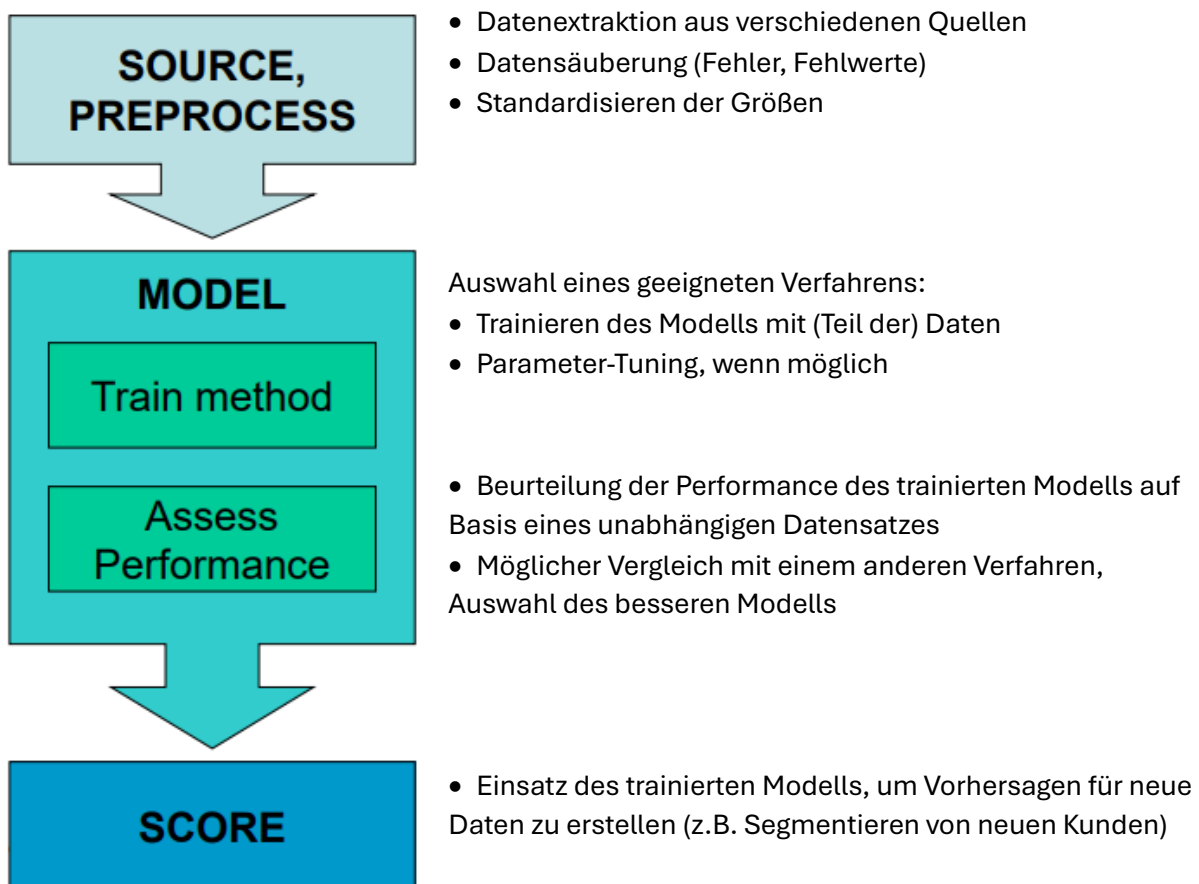
Konzept des maschinellen Lernens

Eine *Maschine* (Computerprogramm) hat aus Erfahrung E für eine Aufgabe T gelernt, wenn T nach Erlernen von E hinsichtlich einer Performance P besser bewältigt wird:

Performance(Task|Experience) > Performance(Task)

Das Ziel des *Lernenden* (=Algorithmus) ist es, aus Informationen zu *Beispielen* oder Instanzen (=Fälle), die durch *Features* bzw. Attribute (=Größen) beschrieben sind, ein reales Phänomen anhand einer *Hypothese* (=Modell) näherungsweise zu erklären.

Workflow für prädikative Methoden



Definition der Datenpunkte

Wichtig für alle Weitere: in allen Machine Learning-Verfahren (einschließlich neuronaler Netze, siehe letztes Modul!) werden Daten als **n-dimensionale** Vektoren beschrieben. Diese nennt man **Merkmals- bzw. Feature-Vektoren**.

Teil des **Preprocessings** (siehe oben) ist es, möglich treffsichere Features zu finden, nämlich:

- Für die Anwendung relevante Informationen codieren
- Nicht zu viele/wenige Dimensionen, keine redundanten/irrelevanten Werte

Beispiele für Anwendungen und mögliche Merkmale:

- Anwendung „Filme nach Genre klassifizieren“: beteiligte DarstellerInnen, Regisseurln, Länge, Produktionsländer, ...
- Anwendung „Handelsklasse von Äpfeln vorhersagen“ (siehe später): Größe, Farbe
- Anwendung „Bilder in Kategorien einteilen“: Farbwerte der einzelnen Pixel

Typische Machine Learning Aufgaben

Überwachtes Lernen

Prozess der Erstellung eines Modells auf Basis eines definierten Lerndatensatzes.

- **Klassifikation:** Labels sind kategorial (binär oder mehrere Klassen). Beispiel: Genre von Filmen anhand von Mitwirkenden vorhersagen
- **Regression:** ähnlich, aber Labels sind *kontinuierliche Werte*. Beispiel: Vorhersage von Verkaufszahlen

Unüberwachtes Lernen

Erstellen eines Modells ohne Referenzdatensatzes und mit dem Ziel, Muster zu finden.

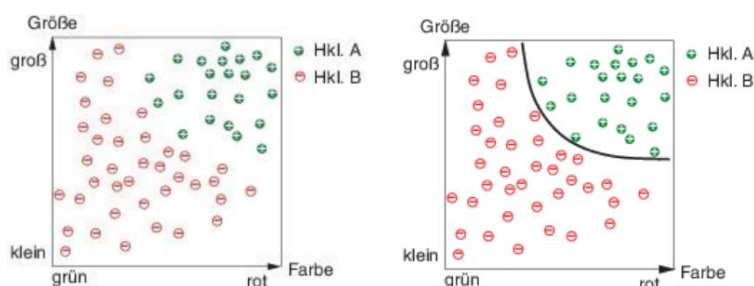
- **Clustering:** Identifikation von ähnlichen Objekten. Beispiel: Bilder zu Alben zusammenfassen
- **Assoziationsregeln:** Finden von zusammengehörigen Objekten. Beispiel: Hinweise der Art „Leute, die dies kauften, kauften auch das“

Beispiel für Klassifikation

Klassifikationsaufgabe: Äpfel basierend auf den **Merkmalen (Features) Größe und Farbe** in Handelsklasse A und B sortieren.

Ein System, das Instanzen basierend auf Merkmalsvektoren automatisch in die Klassen einteilt, nennt man **Classifier**.

Größe [cm]	8	8	6	3	...
Farbe	0,1	0,3	0,9	0,8	...
Handelsklasse	B	A	A	B	...



Maschinelles Lernen

Man unterscheidet:

- **Überwachtes (supervised) Lernen:** Das System hat eine große Menge an **klassifizierten Trainingsdaten** und lernt daraus, auch neue (bislang nicht klassifizierte) Instanzen richtig einzuordnen.
- **Unüberwachtes (unsupervised) Lernen:** Es gibt **keine** Trainingsdaten. Das System erkennt Muster und Ähnlichkeiten in Daten, ohne Referenzwerte zu haben.

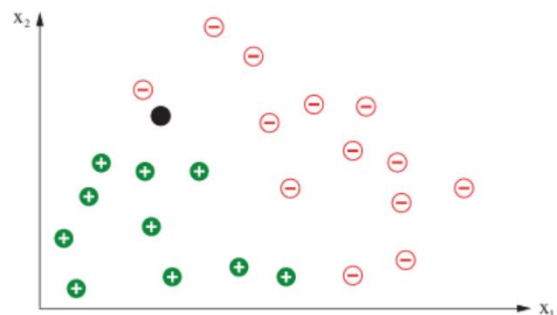
Man kann meist nicht frei auswählen, was man verwendet! Es kommt auf die Anwendung an.

2. Überwachtes Lernen

Nearest Neighbor (NN)

Grundlegende Idee: Neuer Datenpunkt wird anhand des nächstgelegenen Punktes klassifiziert. Wichtige Aufgabe: **Ähnlichkeit** identifizieren (geringer Abstand im Merkmalsraum). Dafür kann z.B. die **euklidische Norm** verwendet werden:

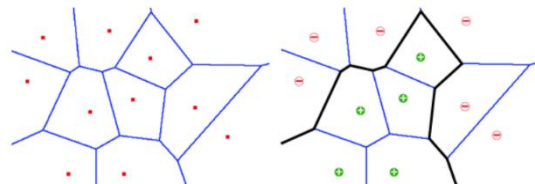
$$d(x, y) = |x - y| = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$



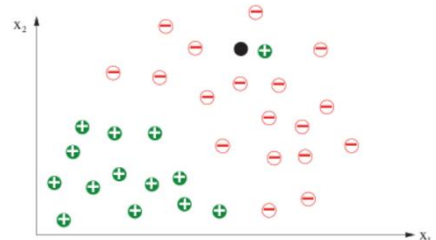
Es gibt jedoch auch viele andere Abstandsmaße.

Analyse von NN mittels Voronoi Diagramm

Voronoi-Diagramm: Jeder Datenpunkt ist von einem konvexen Polygon umgeben, welches das Gebiet um diesen definiert. Die NN-Klassifikation erlaubt Trennung von Klassen jenseits von Linien.



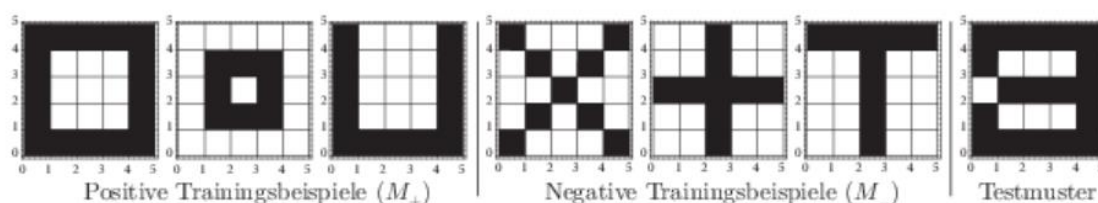
Ein Problem ist die **Überanpassung (Overfitting)**: liegt in der Nähe eines neuen Punktes ein „falscher“ Nachbar, können neue Daten falsch klassifiziert werden (Fehlanpassung an zufällige Fehler/Tauschen).



k-Nearest Neighbor (kNN)

NN ist problematisch, wenn der nächste Nachbar ein Ausreißer ist. Deshalb gibt es beim kNN-Verfahren eine Mehrheitsentscheid unter den k nächsten Nachbarn.

Beispiel: Mustererkennung – binäre Muster mit 5x5 Pixel



Berechne Distanz zu allen Trainingsbeispielen. Mögliches Distanzmaß: **Hamming-Abstand** zweier Bit-Vektoren (Anzahl der unterschiedlichen Bits -> Bitflips). Klasse der k nächsten ergibt Klasse der Eingabeinstanz.

Erweiterung (k)NN für mehrere Klassen

(k)NN-Klassifikation lässt sich problemlos auch **auf mehr als zwei Klassen** anwenden -> die Klasse des neuen Merkmalsvektors wird durch die Klasse des nächsten bzw. der k nächsten Nachbarn bestimmt.

Wird die Zahl der Klassen zu groß, ist es meist nicht mehr sinnvoll, Klassifikationsverfahren einzusetzen:

- Zahl der benötigten Trainingsdaten wächst stark
- Gefahr von Overfitting
- Informationen gehen u.U. verloren

Bei numerischen „Klassen“ könnte Regression möglich sein.

Lernen von Entscheidungsbäumen

Entscheidungsbäume sind für die Praxis ein enorm wichtiges Verfahren (effizient und doch einfach). Sie sind auch sehr anschaulich und durch den Menschen leicht zu verstehen, interpretieren und kontrollieren.

Beispiel: Entscheidungsbaum für Skifahrer:

Klassifikationsbaum: Definition, Beispiel

Ein **Klassifikationsbaum** repräsentiert die **Regeln**, um Antwortklassen von den gegebenen Attributen anzuleiten.

Beispiel: Wenn ein Schigebiet weniger als 100km entfernt ist, dann gehen wir heute Schifahren.

An jedem neuen Tag kann die Entscheidung zum Schifahren auf Basis des Klassifikationsbaums abgeleitet werden – der erreichte Knoten bestimmt die Klasse.

Entscheidungsbaum: Definition, Beispiel

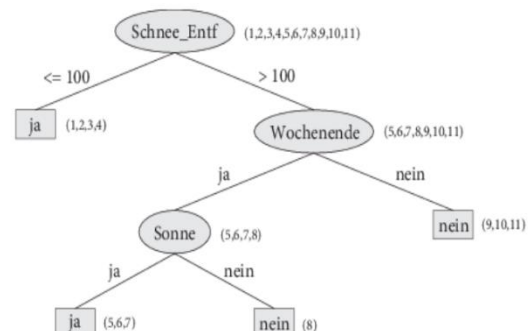
Definition:

- Baum, dessen innere Knoten Merkmale (Attribute) repräsentieren
- Jede Kante steht für Attributwert
- An jedem Blattknoten ist ein Klassenwert angegeben

Daten zum Aufbau des Beispielbaumes (1 Eintrag pro Tag). Manchmal ist wegen widersprüchlichen Daten (z.B. [6] & [7]) nicht in allen Fällen eine korrekte Klassifikation möglich.

Es gibt eine Einschränkung auf Attribute mit endlich vielen Werten.

Variable	Werte	Beschreibung
Skifahren (Zielvariable)	ja, nein	Fahre ich los in das nächstgelegene Skigebiet mit ausreichend Schnee?
Sonne (Merkmal)	ja, nein	Sonne scheint heute?
Schnee_Entf (Merkmal)	≤ 100, > 100	Entfernung des nächsten Skigebiets mit guten Schneeverhältnissen (über/unter 100 km)
Wochenende (Merkmal)	ja, nein	Ist heute Wochenende?



Tag	Schnee_Entf	Wochenende	Sonne	Skifahren
1	≤ 100	ja	ja	ja
2	≤ 100	ja	ja	ja
3	≤ 100	ja	nein	ja
4	≤ 100	nein	ja	ja
5	> 100	ja	ja	ja
6	> 100	ja	ja	ja
7	> 100	ja	ja	nein
8	> 100	ja	nein	nein
9	> 100	nein	ja	nein
10	> 100	nein	ja	nein
11	> 100	nein	nein	nein

Brute Force-Ansatz: alle Bäume erzeugen und die Zahl der Fehlklassifikationen auf den Daten berechnen -> minimale Fehlerzahl, aber zu hohe Rechenzeit.

Heuristischer Algorithmus: den Baum rekursive (angefangen von der Wurzel) aufbauen:

- In jedem Schritt wird nach dem *Attribut mit höchstem Maß an Unreinheit* geteilt (z.B. Schnee_entf <= bzw. > 100)
- Attribut hat „hohe Unreinheit“ wenn nach dem Split im linken und rechten Teilbaum möglichst viele Instanzen dieselbe Klasse haben

Genaue Formalisierung basiert auf mathematischer Definition von „Unreinheit“ (z.B. Entropie, Gini-Koeffizient).

Pruning – der Baumschnitt

Ockhams Rasiermesser (Occam's Razor): Bei gleichwertigen Theorien die einfachere bevorzugen.

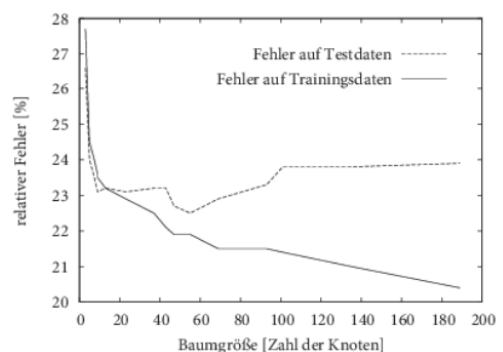
Umgelegt auf Entscheidungsbäume ergibt das: Existieren mehrere mit gleich guter (oder ähnlicher) Fehlerrate, wähle den einfacheren (kleineren).

Ziel eines Verfahrens: Erzeugen eines möglichst kleinen/einfachen Entscheidungsbaums -> einfachere Modelle generalisieren besser, sind einfacher auf neue Daten übertragbar und sind performanter.

Qualitätsbetrachtungen

Fehlerrate (Qualitätsmaß für den Baum): Erstellen mit Trainingsdaten (z.B. 70% der Daten) -> validieren mit Testdaten:

- Fehler auf Trainingsdaten nimmt monoton ab mit Größe des Baumes, nicht aber unbedingt auf den Testdaten
- Beispiel: ab 55 Knoten steigt die Fehlerrate wieder (wie beim NN-Verfahren heißt der Effekt **Overfitting** bzw. Überanpassung)
- Mögliche Lösung: Abbruch des Lernens bei 55 Knoten



Gütekriterien von (supervised) ML-Verfahren

Wie beurteilt man die Qualität eines gelernten Modells? Überblick über gängige Metriken:

		True condition			
		Condition positive	Condition negative	Prevalence = $\frac{\sum \text{Condition positive}}{\sum \text{Total population}}$	Accuracy (ACC) = $\frac{\sum \text{True positive} + \sum \text{True negative}}{\sum \text{Total population}}$
Predicted condition	Predicted condition positive	True positive, Power	False positive, Type I error	Positive predictive value (PPV), Precision = $\frac{\sum \text{True positive}}{\sum \text{Predicted condition positive}}$	False discovery rate (FDR) = $\frac{\sum \text{False positive}}{\sum \text{Predicted condition positive}}$
	Predicted condition negative	False negative, Type II error	True negative	False omission rate (FOR) = $\frac{\sum \text{False negative}}{\sum \text{Predicted condition negative}}$	Negative predictive value (NPV) = $\frac{\sum \text{True negative}}{\sum \text{Predicted condition negative}}$
		True positive rate (TPR), Recall, Sensitivity, probability of detection = $\frac{\sum \text{True positive}}{\sum \text{Condition positive}}$	False positive rate (FPR), Fall-out, probability of false alarm = $\frac{\sum \text{False positive}}{\sum \text{Condition negative}}$	Positive likelihood ratio (LR+) = $\frac{\text{TPR}}{\text{FPR}}$	Diagnostic odds ratio (DOR) = $\frac{\text{LR+}}{\text{LR-}}$
		False negative rate (FNR), Miss rate = $\frac{\sum \text{False negative}}{\sum \text{Condition positive}}$	Specificity (SPC), Selectivity, True negative rate (TNR) = $\frac{\sum \text{True negative}}{\sum \text{Condition negative}}$	Negative likelihood ratio (LR-) = $\frac{\text{FNR}}{\text{TNR}}$	F1 score = $\frac{2}{\frac{1}{\text{Recall}} + \frac{1}{\text{Precision}}}$

Wahrheitsmatrix (Confusion Matrix)

Klassifikationsbeispiel: Modell zur Einordnung von kranken (links) und gesunden (rechts) Menschen. Grün: korrekt bewertete Fälle, Rot: falsch bewertete Fälle.

Wie schaut die Wahrheitsmatrix (WM) für das Beispiel aus?

	Person ist krank ($r_p + f_n$)	Person ist gesund ($f_p + r_n$)
Test positiv ($r_p + f_p$)	richtig positiv (r_p)	falsch positiv (f_p)
Test negativ ($f_n + r_n$)	falsch negativ (f_n)	richtig negativ (r_n)

Sensitivität (true/positive)

Auch Richtig-positiv-Rate, Empfindlichkeit oder Trefferquote (Recall) genannt.

Ist der Anteil der korrekt als positiv klassifizierten Objekte an der Gesamtheit der tatsächlich positiven Objekte.

$$P(\text{positives Testergebnis} | \text{tatsächlich krank}) = \frac{r_p}{r_p + f_n}$$

Falsch-negativ-Rate

Anteil der fälschlich als negativ klassifizierten Objekte an der Gesamtheit der tatsächlich positiven Objekte.

$$P(\text{negatives Testergebnis} | \text{tatsächlich krank}) = \frac{f_n}{r_p + f_n}$$

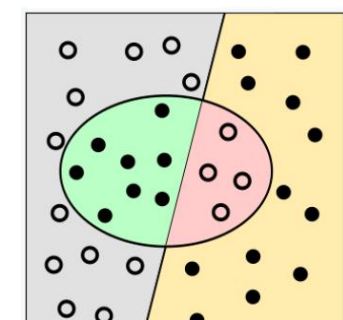
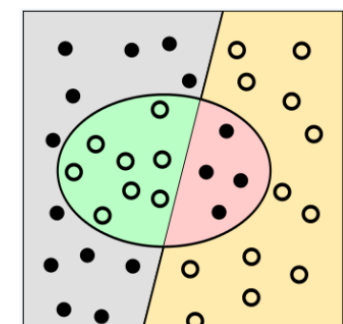
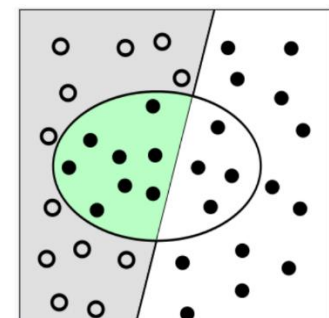
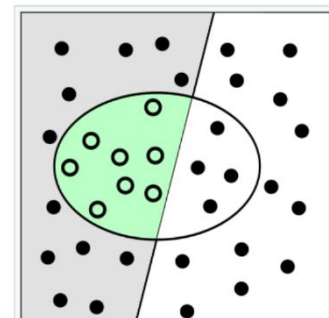
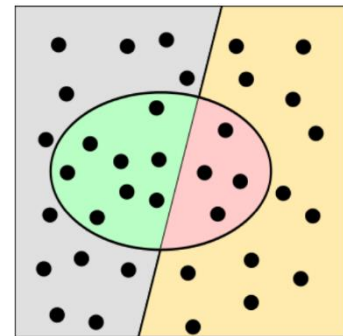
Korrekt- und Falschklassifikationsrate

Klassifikationsrate (Treffergenauigkeit, **Accuracy**): Anteil aller Objekte, die korrekt klassifiziert wurden (=Hauptdiagonale der Wahrheitsmatrix zur Gesamtheit).

Klassifikationsfehler: falsch klassifizierte Objekte zur Gesamtheit (außerhalb der Hauptdiagonale).

$$P(\text{richtig klassifiziert}) = \frac{\text{richtig klassifiziert}}{\text{alle Fälle}} = \frac{r_p + r_n}{r_p + f_p + r_n + f_n}$$

$$P(\text{falsch klassifiziert}) = \frac{\text{falsch klassifiziert}}{\text{alle Fälle}} = \frac{f_p + f_n}{r_p + f_p + r_n + f_n}$$



3. Unüberwachtes Lernen

Clustering

Erstes Beispiel für unüberwachtes maschinelles Lernen (die Trainingsdaten sind nicht gelabelt/klassifiziert).



Das Ziel ist das Finden von Strukturen in Trainingsdaten. In Cluster ist der Abstand benachbarter Punkte kleiner als der Abstand zwischen Punkten verschiedener Cluster (-> geeignetes Abstandsmaß festlegen).

k-Means

Wenn die Zahl der Cluster vorab bekannt ist, kann das *k*-Means Verfahren angewendet werden (k Cluster mit den Mittelpunkten $\mu_1, \dots, \mu_k$).

Vorgehensweise:

1. Klassifikation aller Daten zum nächsten Clustermittelpunkt
2. Neuberechnung der Clusterpunkte

$K\text{-MEANS}(\mathbf{x}_1, \dots, \mathbf{x}_n, k)$

initialisiere $\mu_1, \dots, \mu_k$ (z.B. zufällig)

Repeat

Klassifiziere $\mathbf{x}_1, \dots, \mathbf{x}_n$ zum jeweils nächsten μ_i

Berechne $\mu_1, \dots, \mu_k$ neu

Until keine Änderung in $\mu_1, \dots, \mu_k$

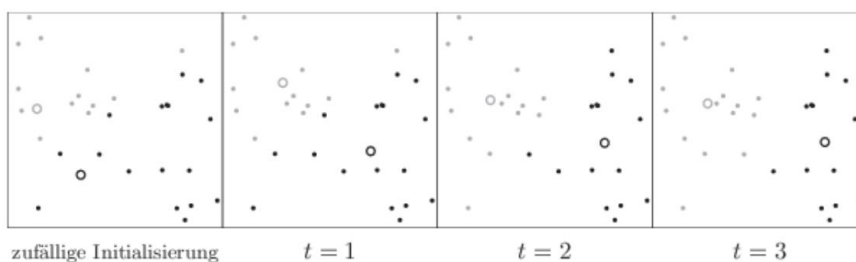
Return $(\mu_1, \dots, \mu_k)$

$$\mu = \frac{1}{l} \sum_{i=1}^l \mathbf{x}_i$$

Berechnung der Clustermittelpunkte:

k-Means mit $k=2$

Nach drei Iterationen stabilisieren sich die Anfangs zufällig gewählten Klassenzentren ($n=30$ Datenpunkte):



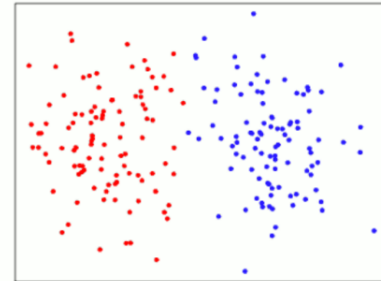
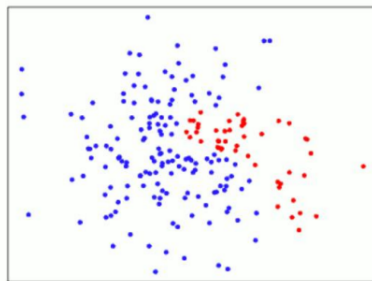
Es gibt *keine Konvergenzgarantie*, das Verfahren konvergiert aber meistens recht schnell (d.h. die Zahl der Iterationen ist viel kleiner als die Zahl der Datenbanken). Die Komplexität wird mit **O(ndkt)** berechnet (n =Punkte, d =Dimension, t =Zahl der Iterationen).

Bewertung eines Clusterings

Wir wollen am Ende wissen wie gut oder schlecht ein Clustering ist.

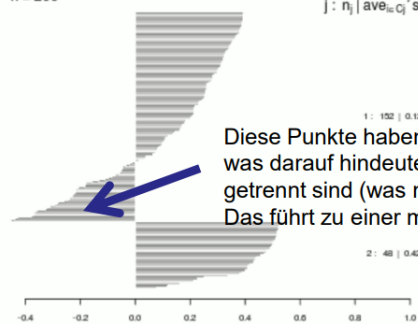
Dazu braucht es ein **Gütekriterium**, eines davon ist das Silhouette-Width-Kriterium: Die Idee hinter diesem ist, dass Datenpunkte möglichst gut im eigenen Cluster, und möglichst weit weg von fremden Clustern liegen (lässt sich mathematisch genau definieren).

Silhouette-Width



Silhouette plot of (x = c2, dist = d)
n = 200

2 clusters C_j
j : n_j | $ave_{i \in C_j} S_i$

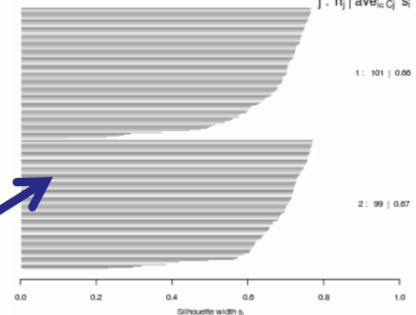


Diese Punkte haben schlechte Werte, was darauf hindeutet dass die Cluster schlecht getrennt sind (was man in der Grafik auch sieht). Das führt zu einer mäßigen Gesamtbewertung.

Hier haben alle Punkte recht gute Werte → bessere Gesamtbewertung.

Silhouette plot of (x = c2, dist = d)
n = 200

2 clusters C_j
j : n_j | $ave_{i \in C_j} S_i$



OMRk-Algorithmus

Meistens weiß man im Voraus nicht, was eine gute Clusterzahl ist. Diese kann berechnet werden, indem man k-Means für steigende Clusterzahlen (1 bis n) laufen lässt und bewertet, wie gut das Clustering jeweils ist (anhand Silhouette-Width).

Dieser Ansatz führt zum OMRk-Algorithmus (Ordered Multiple Runs of k-Mean):

OMRK($x_1, \dots, x_n, p, k_{max}$)

$S^* = -\infty$

For $k = 2$ To k_{max}

For $i=1$ To p

Erzeuge eine zufällige Partition mit k Clustern (Initialisierung)

Ermittle mit k-Means eine Partition P

Bestimme S für P

If $S > S^*$ Then

$S^* = S$; $k^* = k$; $P^* = P$

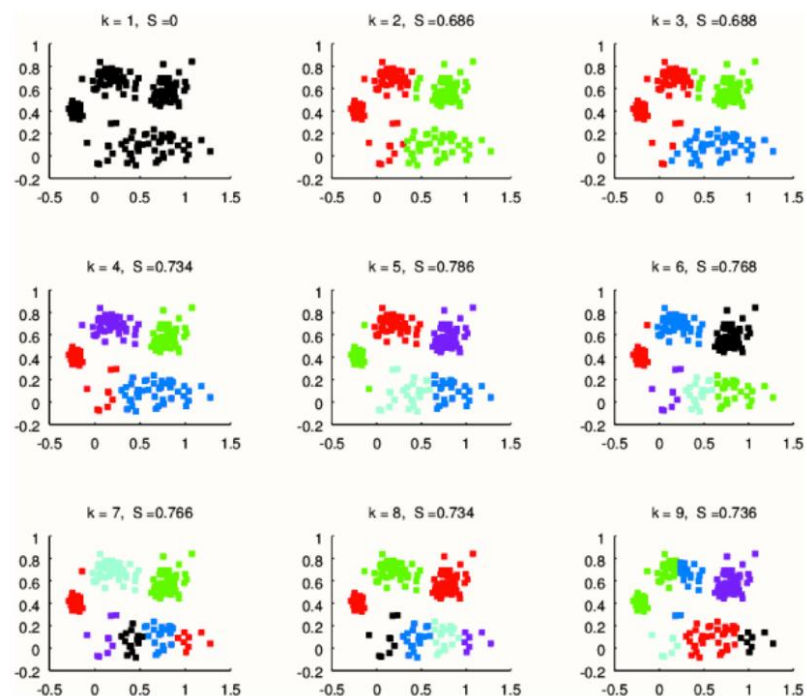
Return (k^*, P^*)

Beispiel: OMRk-Algorithmus

Ergebnisse für
k=2 bis 9

Bester Wert mit
S=0.786 und k=5

Auge würde k=4
präferieren (höhere
Dichte in Clustern)



Eckpunkte des Moduls

Maschinelles Lernen umfasst **überwachte** und **unüberwachte Lernverfahren** sowie **Lernen durch Verstärkung**. In der Regel kommen auch **Data Mining** Techniken zum Einsatz, um Semantik aus Rohdaten zu gewinnen.

Klassifikation meint das **Zusammenfassen von Objekten zu Klassen** und wird typischerweise mittels **überwachten Lernverfahren** erstellt. Klassifikationsverfahren umfassen unter anderem Methoden wie **k-Nearest-Neighbor** oder **Entscheidungsbäume**. Solche Modelle unterliegen **Fehlklassifizierungen** genauso wie dem Phänomen der **Überanpassung (Overfitting)**.

Bei **unüberwachten Lernverfahren** gibt es keine Label-Daten und auch kein Unterteilen des Datensatzes für Training und Validierung.

Im Rahmen von Clustering ist entscheidend, ein gutes **Abstandsmaß** zu wählen. Mittel **k-Means** können Datenpunkte im n-dimensionalen Merkmalsraum in effizienter Weise geclustert werden.

Die optimale Anzahl an Clustern kann entweder geschätzt oder durch ein **Verfahren zur Bewertung eines Clusterings** herausgefunden werden.

Modul 4: Neuronale Netze

1. Grundlagen

Grundlegende Idee

Die grundlegende Idee von neuronalen Netzen ist die **Imitation des menschlichen Gehirns**. Das Gehirn ist ein Verbund von 10 bis 100 Milliarden Nervenzellen, die Neuronen heißen.

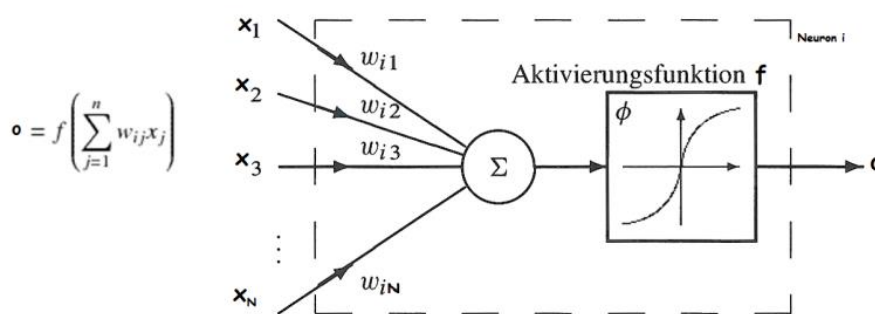
1943 veröffentlichten McCulloch & Pitt einen Beitrag über ein **mathematisches Modell des Neurons**. Dieses Modell ist die Basis von künstlichen neuronalen Netzen.

Bionik-Zweig in der KI: Modellierung und Simulation von neuronalen Netzen -> wenig Ähnlichkeit mit anderen AI-Ansätzen wie Logik oder probabilistischem Schließen.

Mathematisches Modell (Neuron)

Der **Merkmal-/Feature-Vektor** eines Datenpunktes ist der **Eingabevektor** $x_1, \dots, x_n$.

Neuron i **gewichtet** jedes Feature mit einem **Gewicht** w_{ij} , summiert diese auf, und wendet eine Aktivierungsfunktion f an.



Der **Eingabevektor** wird so auf einem **Ausgabewert** o abgebildet.

Künstliche Neuronen werden auch „Units“ genannt.

Es sind unterschiedliche Aktivierungsfunktionen möglich, z.B.:

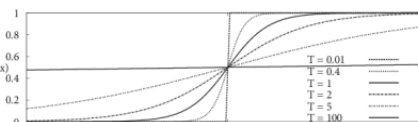
- **Identität:** $f(x) = x$ (in der Regel wegen Konvergenzproblemen ungünstig)

$$H_{\Theta}(x) = \begin{cases} 0 & \text{falls } x < \Theta \\ 1 & \text{sonst} \end{cases} \quad x_i = \begin{cases} 0 & \text{falls } \sum_{j=1}^n w_{ij}x_j < \Theta \\ 1 & \text{sonst} \end{cases}$$

- **Heavisidesche Stufenfunktion:**

$$f(x) = \frac{1}{1 + e^{-\frac{x-\Theta}{T}}}$$

- **Sigmoid-Funktion:**

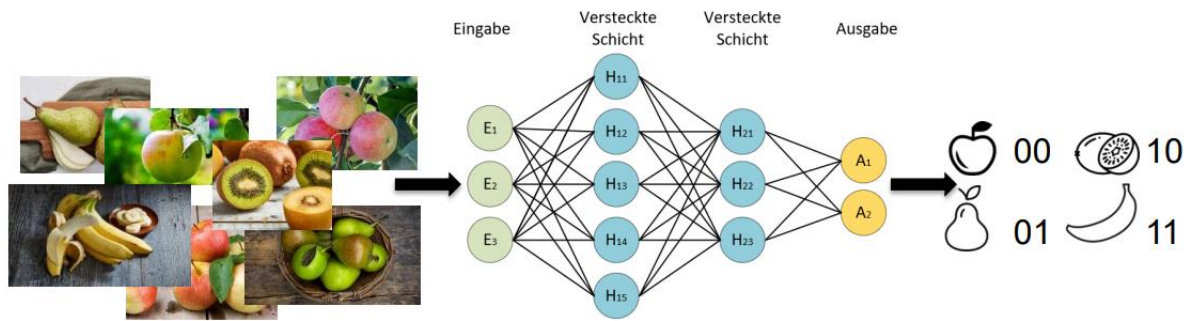


Die geeignete Funktion ist anwendungsabhängig (Eingabedaten müssen je nach Aktivierungsfunktion ggf. skaliert werden).

Mathematisches Modell (Neuronales Netz)

Die Ausgabe von Units kann die Eingabe in weitere Units sein.

Neuronales Netz:



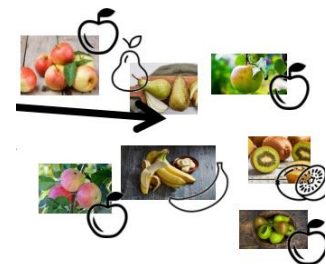
Bevor es um die Details geht:

- Definieren was die Eingabe sein soll (z.B. die Farbwerte der Pixel eines Bildes)
- Definieren was die Ausgabe sein soll (z.B. 2-stellige Binärzahl zur Darstellung von 4 möglichen Klassen – in der Praxis wäre das eher ein eigenes Bit für jede Klasse)

Mathematisches Modell

Die zentrale Aufgabe ist das **Trainieren eines Modells**.

- Man braucht ein **Trainingsset**: Das ist eine große Menge von Eingabewerten und die zugehörigen (gewünschten) Ausgaben
- Ein Trainingsalgorithmus bringt das Netz dazu, zu den Eingaben möglichst die richtigen Ausgaben zu liefern
- Dazu werden die **Gewichte** w_{ij} anhand von Trainingsdaten so gesetzt, dass möglichst viele Eingabevektoren auf die „richtige“ (=gewünschte) Ausgabe abgebildet werden



Das Ziel ist, dass diese Vorhersage auch bei neuen Daten (außerhalb des Trainingssets) funktioniert (-> Dazu muss das Trainingsset repräsentativ sein).



2. Das Perzeptron

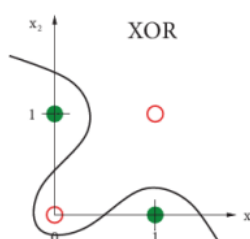
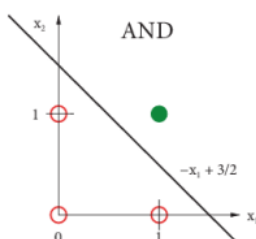
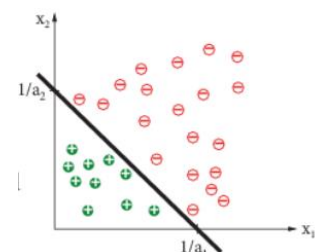
Linearer Klassifizierer (Perzeptron)

Einfacher Fall: Die Datenmenge ist **linear separabel**, dann reicht ein einziges Neuron.

D.h. es existiert eine Gerade, die die beiden Klassen trennt, formal:

$$a_1x_1 + a_2x_2 = 1$$

Beachte: Nicht alle Daten sind linear separabel, z.B. nicht XOR:



Lernregel für das Perzeptron

Das Perzeptron kann durch folgenden Algorithmus trainiert werden:

1. Iteriere wiederholt durch alle Trainingsinstanzen
2. Im Falle einer Fehlklassifikation, addiere bzw. subtrahiere die Instanz des Gewichtsvektors (je nachdem ob positives oder negatives Beispiel)
3. Beende, sobald alle Instanzen richtig klassifiziert sind (bei linear separablen Daten ist garantiert, dass das irgendwann passiert)

Beachte: Der Algorithmus funktioniert, das ist aber nicht selbstverständlich und auch nicht leicht zu sehen! Den Beweis davor gibt es in der tieferen Mathematik.

Beispiel

Beispiel: Trainieren eines Perzeptrons auf den Mengen

$M_+ = \{(0, 1.8), (2, 0.6)\}$

$M_- = \{(-1.2, 1.4), (0.4, -1)\}$

(2 positive, 2 negative Datenpunkte)

PERZEPTRONLERNEN(M_+, M_-)

w = beliebiger Vektor reeller Zahlen

Repeat

For all $x \in M_+$

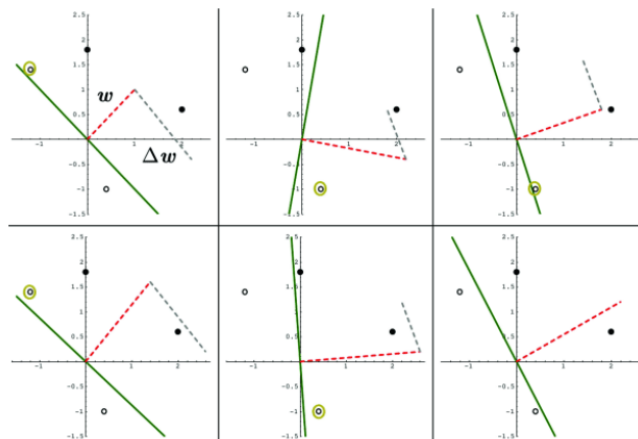
If $w \cdot x \leq 0$ Then $w = w + x$

For all $x \in M_-$

If $w \cdot x > 0$ Then $w = w - x$

Until alle $x \in M_+ \cup M_-$ werden korrekt klassifiziert

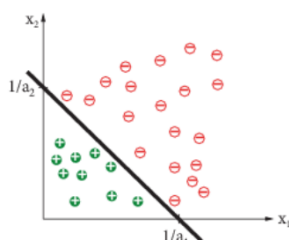
- Rot strichliert = Gewichtsvektor
- Grüne Linie = aktuelle Trenngerade (zwischen Klassen)
- Grau strichliert = zu addierenden Änderungsvektor



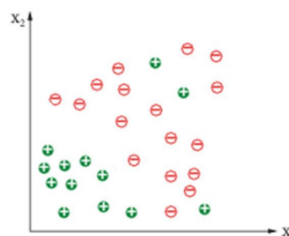
Optimierung und Ausblick

Perzeptron ist das einfachste neuronale Lernverfahren, kann aber **nur linear seperable Klassen** trennen:

Erlernbar:



Nicht erlernbar:



Mehrlagige Netze sind wesentlich mächtiger. Selbst bei linear separablen Klassen konvertiert Perzeptron oft nur langsam.

3. Mehrlagige Neuronale Netze (Feedforward, Gradientenabstieg und Backpropagation)

Lineare Netze mit minimalem Fehler

Gegeben: N Paare von Trainingsvektoren $T = \{(q^1, t^1), \dots, (q^N, t^N)\}$ mit $q^p \in [0, 1]^n$, $t^p \in [0, 1]^m$, q ist dabei jeweils die Eingabe, p die richtige Ausgabe.

z.B. $q =$ , $p = [0, 0]$ (um  zu repräsentieren)

Gesucht: Funktion $f : [0, 1]^n \rightarrow [0, 1]^m$, die den mittleren quadratischen Fehler $\sum_{p=1}^N (f(q^p) - t^p)^2$ (d.h. Abweichung von erwünschter und gelieferter Ausgabe) auf den Daten minimiert.

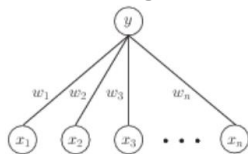
Wir beschränken uns zuerst auf Netze mit nur **zwei Schichten**: 1. Die Eingabe, 2. Eine Neuronenschicht (Eingabe zählt man meist als Schicht mit).

Einblicke in die Mathematik dahinter

Startpunkt: Zweilagiges Netzwerk, bei dem das einzige Neuron y seine Aktivierung mittels

$$y = f\left(\sum_{i=1}^n w_i x_i\right) \text{ berechnet}$$

Aktivierungsfunktion: bei nur einer Schicht ist die Identität $f(x) = x$ ausreichend:



Initialisierung der Gewichte: zufällig, aber niemals mit 0.

Ziel: Berechnung der Gewichte anhand des Trainingssets.

Formal: Suche Vektor $\mathbf{w}$, der den Fehler $E(\mathbf{w})$ minimiert:

$$E(\mathbf{w}) = \sum_{p=1}^N (\mathbf{w} \mathbf{q}^p - t^p)^2 = \sum_{p=1}^N \left(\sum_{i=1}^n w_i q_i^p - t^p \right)^2$$

Dazu lassen sich Gleichungen für die Berechnung der Gewichte aus der Differenzialrechnung konstruieren...

$$\frac{\partial E}{\partial w_j} = 2 \sum_{p=1}^N \left(\sum_{i=1}^n w_i q_i^p - t^p \right) q_j^p = 0 \rightarrow \sum_{p=1}^N \left(\sum_{i=1}^n w_i q_i^p q_j^p - t^p q_j^p \right) = 0 \rightarrow \sum_{i=1}^n w_i \sum_{p=1}^N q_i^p q_j^p = \sum_{p=1}^N t^p q_j^p,$$

...was letztendlich das folgende Gleichungssystem ergibt...

$$A_{ij} = \sum_{p=1}^N q_i^p q_j^p \quad \text{und} \quad b_j = \sum_{p=1}^N t^p q_j^p \quad \text{und} \quad \mathbf{A} * \mathbf{w} = \mathbf{b}$$

...das man noch nach $\mathbf{w}$ löst und das Trainingsset einsetzt.

Das geht sehr tief in die Analysis, die Details sind für uns nicht relevant!

Inkrementelles Lernen

Pseudocode für ein Lernverfahren, die sogenannte **Delta-Regel**:

DELTALERNENINKREMENTELL (*Trainingsbeispiele*, η)

Initialisiere alle Gewichte w_j zufällig

Repeat

For all (q^p, t^p) $\in$ *Trainingsbeispiele*

 Berechne Netzausgabe $y^p = w^p q^p$

$w = w + \eta(t^p - y^p) q^p$

Until w konvergiert

/* Gehe wiederholt über
 ganzes Trainingsset */

/* Aktualisiere Gewichte
 je nach Abweichung vom
 richtigen Ergebnis */

Unterschied Delta-Regel zum Perzeptron-Lernen:

- Man braucht nicht alle Trainingsdaten von Anfang an
- Kommen nachträglich neue rein, kann man einfach weiterlernen, d.h. schrittweise verbessern

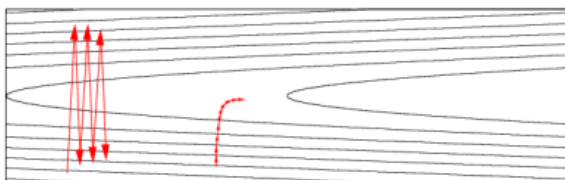
Parameter η der Delta-Regel

Außerdem hat die Delta-Regel einen wählbaren Parameter:

- Parameter: **Lernrate η** (Konstante zwischen 0 und 1)
- Je höher η , umso schneller ändert der Algorithmus Gewichte

Die richtige Lernrate ist wesentlich, aber nicht zu leicht zu bestimmen:

- **Ist sie zu klein, dauert Lernen zu lange** (entspricht einem Menschen der auf seinem Standpunkt beharrt, auch wenn neue Information dagegen spricht)
- **Ist sie zu groß, besteht die Gefahr der Oszillation** (entspricht einem Menschen der bei jedem neu gelesenen Satz bereit ist fast alles zu verwerfen, das er bisher weiß)



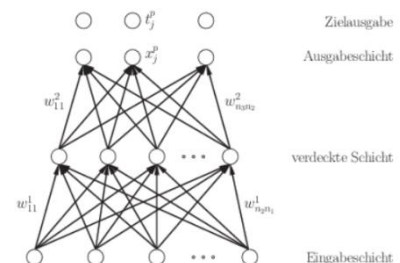
(links zu groß, rechts zu klein)

Backpropagation-Algorithmus

Wie funktioniert das nun für Netze mit mehr als 2 Schichten ->

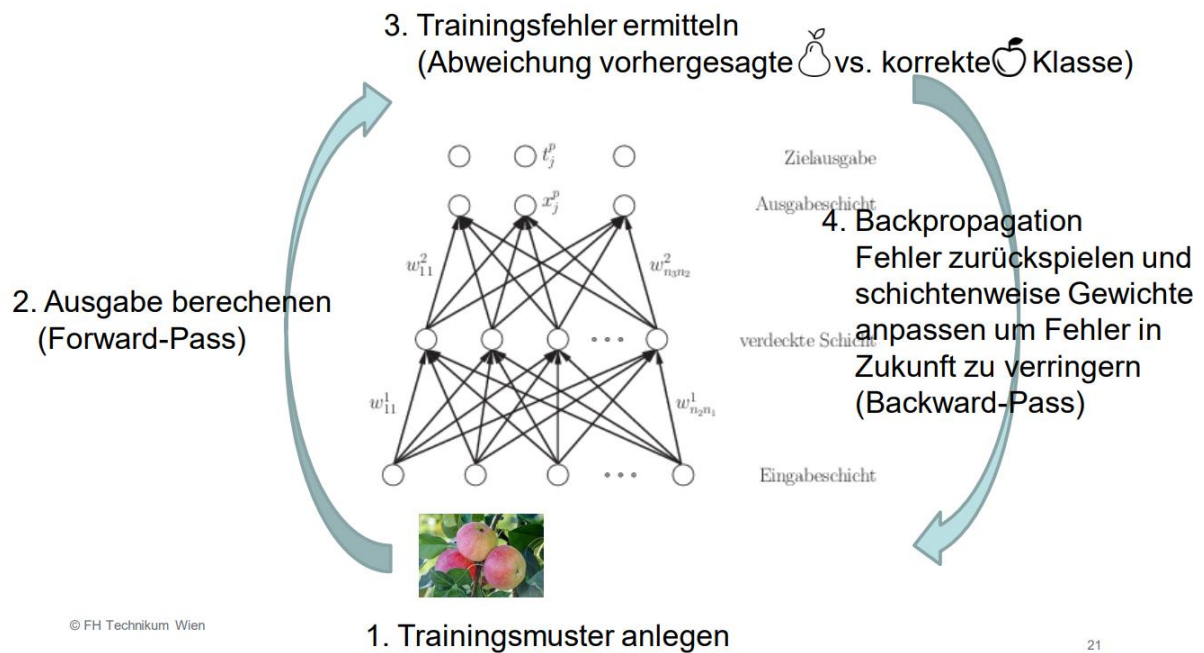
Backpropagation-Algorithmus (das am meisten genutzte neuronale Modell).

Es geht aus der Delta-Regel hervor, es funktioniert aber nur für beliebig viele Schichten -> universelle Einsetzbarkeit für Approximationsaufgaben.



Verwendet die Sigmoid-Aktivierungsfunktion (Identität ist nun nicht mehr geeignet!).

Idee



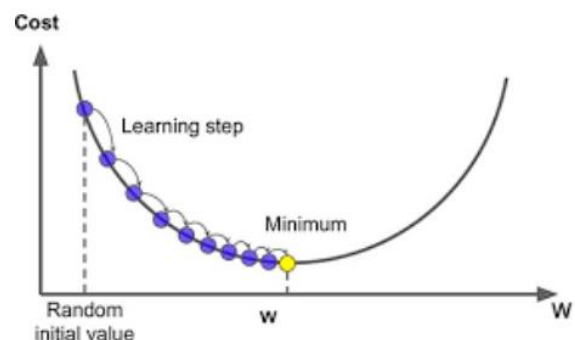
21

Wie passt man Gewichte an? -> Gradientenabstieg („Abstieg vom Berg zum Minimum“).

„Berg“ = aktueller Fehler

Gewichte werden in jedem Schritt so angepasst, dass Fehler über das Trainingsset kleiner wird.

Grundlage ist hier wiederum die Analysis: Ableitungen der Fehler-Funktion zeigen, in welche Richtung sie kleiner wird.

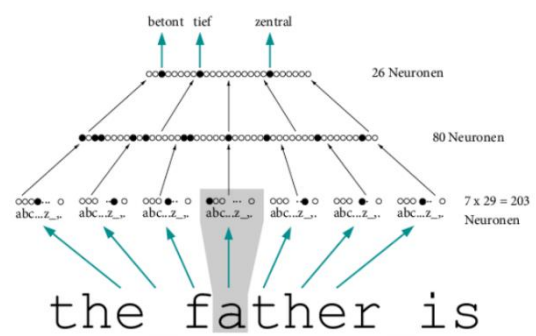


Anwendungsbeispiel: NETtalk

NETtalk (1986!): System, das in der Lage ist, einen Text in englischer Sprache laut und verständlich vorzulesen.

Eingabeschicht:

- $7 \times 29 = 203$ Neuronen (aktueller Buchstabe und 3 davor/danach; „a...z<space>,.“)
- 80 verdeckte Neuronen
- 26 Ausgabeneuronen (Ausspracheattribute)
- Training mit 1000 Wörtern (zufällig buchstabenweise nacheinander)



Software rund um Neuronale Netze

Es gibt heutzutage viele Softwarebibliotheken zum Trainieren und Anwendungen neuronaler Netze.

Momentan ist **TensorFlow** von Google besonders populär (darauf aufbauend **Keras**).

In Produktivumgebungen werden meistens **GPUs** (Grafikkarten) statt CPUs verwendet, da diese eine bessere Parallelisierbarkeit besitzen.

Alternativen für andere Sprachen:

- **Accord.NET** oder **ML.NET** für die .NET-Plattform
- **Deeplearning4j** für Java
- Pakete für **Mathematica** und **MATLAB**

4. Deep Learning und Anwendungen

Fluch der Dimension

Finden von geeigneten Merkmalen ist häufig nicht automatisch. Meist hat man sehr viele Dimensionen. Einfach alle verfügbaren in das Netz werfen funktioniert meist nicht gut -> „Fluch der Dimensionen“.

Grund: Redundanzen, irrelevante Features -> „NN sieht vor lauter Bäumen den Wald nicht“.

Beispiel Bilderkennung: Ein Bild in HD: 10 Mio Pixel mit je 3 Byte (RGB) pro Bildpunkt -> 3 Mio Dimensionen, aber benachbarte Pixel sind in der Regel ähnlich gefärbt und nur in Kombination aussagekräftig.

Aufgabe des **Knowledge Engineeres** daher: Finden von möglichst wenigen aber aussagekräftigen Merkmale, die als Eingabe für Lernalgorithmen geeignet sind.

Mögliche Ansätze

- **Händische Merkmalsauswahl**: Sehr arbeitsaufwändig
- **Hauptkomponentenanalyse** (principal component analysis): Algorithmische Reduktion der Dimension des Vektorraums in Richtung der Dimension mit höchster Varianz
- **Deep Learning**: Seit 1995
 - Verwendung von Netzen mit „vielen“ Schichten: **Deep Networks**
 - Input besteht tatsächlich aus den *vollständigen* Daten
 - Erste Schichten sollen darin die relevanten Features finden
 - Späteren Schichten lernen auf den wenigen, relevanten Features
 - Aber: Wegen hoher Anzahl an Schichten sehr **rechenintensiv**
 - **Braucht teure Hardware**

Deep Learning

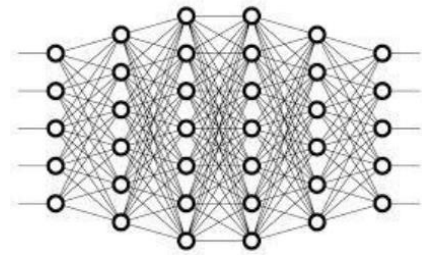
Deep Learning = Trainieren von Deep Networks

Deep Networks = spezieller Typ von Neuronalen Netzen

Deep Network ist schlicht ein Name für neuronale Netze mit „vielen“ Schichten (keine einheitliche Definition wie viele Schichten ein NN zu einem Deep Network machen). Durchaus üblich sind aber 20 oder mehr.

Nach Typen der Schichten weitere Unterscheidung, u.a. in:

- Convolutional Neural Networks (CNNs)
 - Z.B. Bilderkennung
- Deep Belief Networks (DBNs)
 - Z.B. für Bilderzeugung
- Recurrent Neural Networks (RNNs)
 - Z.B. für Spracherkennung



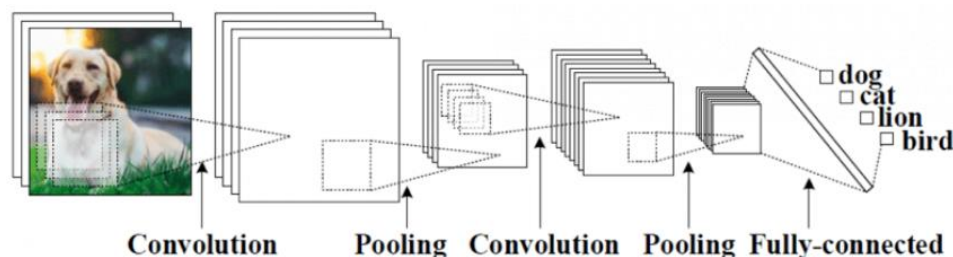
Convolution Neural Network (CNN)

UL läuft über alle Merkmale (=sehr aufwendig).

Merkmalsschichten sind auch nicht vollständig, sondern nur über wenige Neuronen mit vorheriger Schicht vernetzt.

Abwechselnd Faltungs- (convolution) und Pooling-Schichten:

- In der Faltungsschicht wird ein trainierbarer linearer Filter angewendet
- In der Pooling-Schicht wird von Eingabeneuronen der Mittelwert oder das Maximum berechnet



Deep Learning in der Praxis

Beispieldatensatz: MNIST Datensatz (60.000 Instanzen handgeschriebener Zahlen 0-9, 10.000 Einträge in den Testdaten).



Gängigste Bibliotheken: Tensorflow/Keras (Python).

Wird z.B. von Google Streetview verwendet.

Anwendungen – Kreativität

Durch (überraschend kleine) Adaption der Algorithmen kann Deep Learning nicht bloß für die **Klassifikation**, sondern auch für das **kreative Erzeugen** von Daten verwendet werden.

Mathematischer Ansatz (Grundidee):

- Training des NN mit historischen Daten der jeweiligen Kunstform
- Lege zufallsgenerierten Eingabevektor an
- Ändere im Training nicht (wie bei Klassifikation) die Gewichte, sondern den Eingabevektor, um den Fehler zu minimieren

Beispiel 1: Jazz Musik nach einem bestimmten Stil komponieren

Beispiel 2: Text im Stil von Shakespeare dichten

Beispiel 3: Bilder anhand von Text erstellen

Beispiel 4: Kunststil anpassen:

- Nehmen wir 5 Stilrichtungen (Maler), das sind die kleinen Bilder in B-F
- Trainiere 5 Deep NNs für diese Richtungen
- Wir wollen nun ein neues Bild (A) in diesen 5 Stilen darstellen



Vorgehensweise:

- UL des CNN: 16 Convolution-, 5 Pooling-Layer
- Anlegen von weißem Rauschen
- Input per Gradientenabstieg so lang verändern, bis Fehler minimal sind
- Ergebnis: die großen Bilder B-F

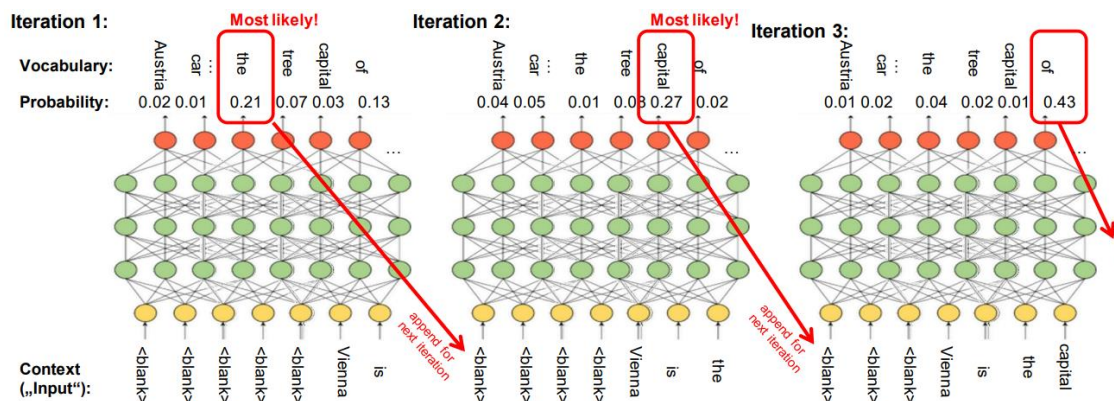
Anwendung: ChatGPT (und ähnliche LLMs)

ChatGPT ist ein von OpenAI entwickelter und 2022 veröffentlichter Chatbot. Es ist ein sogenanntes *Large Language Model (LLM)* basierend auf Generative Pretrainer Transformer (GPT), einer speziellen Form von Deep Neural Networks.

Grundidee: Vorhersage der wahrscheinlichsten Antwort.

Trainingsdaten: Milliarden Websites (inkl. Wikipedia) und Bücher.

GPT: „Vervollständigung“ eines Textes durch wahrscheinlichstes nächstes Wort (bzw. Token):



Chatversion: sehr vereinfacht gesagt den gesamten Chatverlauf inkl. Namen der zwei Chatpartner als zu vervollständigenden Text (in der Praxis ist das komplexer).

Der Chatbot ist sehr gut und hat auch die Fachwelt überrascht. Genau da liegen aber auch **Gefahren:**

- Antworten sind **nicht** zwingend richtig!
- Vom Grundansatz her sind Fehler nie ganz auszuschließen
- Nur weil der Chatbot meistens gute Antworten gibt, darf man diesen nicht blind vertrauen -> immer nachdenken!
- Besonders gefährlich: Auch sprachlich gut formulierte Antworten, die auf den ersten blick auch inhaltlich richtig erscheinen, können trotzdem falsch sein.

Fazit: Neuronale Netze

Lernen **Input-Output-Mapping** aus Daten. Neuronale Netze sind Teilgebiet des ML, aber sind wegen Komplexität und Erfolg auch eigenes Forschungsfeld.

Herausforderungen:

- Rechenaufwand
- Mögliches Overfitting
- Entscheidungen nur mit bestimmter Wahrscheinlichkeit richtig
- Entscheidungen nicht erklärbar (im Gegensatz z.B. zur Logik)

Anwendungen:

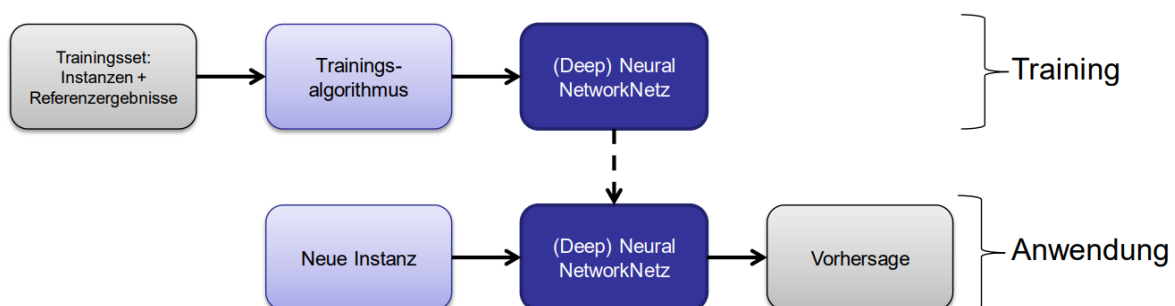
- Gesichts- und Emotionserkennung
- Merkmalerkennung in Audio-, Bild- und Videodaten
- Prüfen der Kreditwürdigkeit von Bankkunden
- Vorhersagen von Aktienkursen, u.v.m.

5. Reinforcement Learning und Anwendungen

Reinforcement Learning

Gewöhnliches NN-Learning braucht ein **Trainingsset**, also eine große Menge an: Instanzen + Referenzergebnisse (gewünschte Ergebnisse).

NN lernt Muster und überträgt sie auf neue Instanzen.



Was tun, wenn nicht ausreichend Trainingsdaten vorhanden sind? -> **Reinforcement Learning (RL)** bzw. **Lernen durch Verstärkung.**

Beim RL benötigt NN keine Trainingsdaten, aber einen **Trainer** der Entscheidungen des NN im **Nachhinein** beurteilt.

The diagram illustrates the training loop of a Reinforcement Learning system. It consists of three main components in a sequence: a dark blue box labeled "(Deep) Neural Network", a light gray box labeled "Aktion", and a light blue box labeled "Trainer". An arrow labeled "Zufällige Initialisierung" points into the "Neural Network" box. An arrow points from the "Neural Network" box to the "Aktion" box, and another from "Aktion" to "Trainer". A feedback loop arrow labeled "Reward (Belohnung oder Bestrafung)" points from the "Trainer" box back to the "Neural Network" box. A bracket on the right side groups the "Aktion" and "Trainer" boxes under the label "Training während Anwendung(*)".



- Reinforcement Learning ist aus Sicht von Neuronalen Netzen eine **Anwendung**: RL baut selbst auf (in der Regel: Deep) NNs auf
- Andererseits ist RL immer noch recht allgemein und kann auch als **Methode** für konkrete Anwendungen gesehen werden

- Agent bekommt Screenshot als Input
- Agent gibt zu drückende Taste(n) aus
- Feedback/Reward (spielabhängig)
 - Z.B. positiv wenn Gegner getroffen, negativ wenn selbst getroffen worden

Beachte: Es ist nicht direkt einprogrammiert, dass das X dem O nachzulaufen hat (wäre trivial, aber nicht AI!). Die AI soll selbst darauf kommen und bekommt dazu einen Reward. Dieser Reward ist *invers proportional zum Abstand* zwischen X und O, also *umso höher, je näher sie sind*.

- Vorbild für Neuronale Netze: Die Natur und **das menschliche Gehirn**.
- Wichtigste Klasse der neuronalen Netze: **Feed Forward Netze** – Daten zum Training oder zur Anwendung an Eingangsschicht anlegend und Ausgang erheben.
- Konkrete Ansätze wie **Perzeptron**, **Methode der kleinsten Quadrate**, **Delta-Regel**, **Backpropagation** und komplexe Varianten wie diverse **Deep Learning** Verfahren
- **Reinforcement Learning** generiert Trainingsdaten selbst „on-the-fly“, basierend auf Information, ob die Entscheidung gut war



Übungsaufgaben/Alte Tests

Abschlussprüfung Haupttermin 2023

1. Grundlagen (24 Punkte)

a) *Nennen Sie 3 Anwendungen der AI und beschreiben Sie diese kurz.*

Spracherkennung: Systeme, die in der Lage sind, gesprochene Wörter aufzuzeichnen und zu erkennen. Dies benötigt eine Kombination aus unterschiedlichsten Verfahren auf Hard- und Softwareebene (Digitalisierung des Audiosignals, Mustererkennung, linguistische Datenverarbeitung).

Beispiel: Intelligenter persönlicher Assistent (Smartphone, Echo,...) -> Sprachinput führt zu Aktionen (Informationen liefern, Signale senden, ...)

Maschinelles Sehen (Bildanalyse): Systeme die Bilddaten und Videos verarbeiten und (unter anderem) versuchen, die Fähigkeiten der menschlichen visuellen Wahrnehmung zu imitieren. Typischerweise wird hier ML und Data Mining verwendet.

Anwendungsfälle: Mustererkennung/Merkmalsextraktion (Gesichtserkennung, Objekterkennung).

Schwierigkeiten: Unterdrückung der Umgebungsbedingungen, Erkennung von abstrakten Konzepten (Emotionen, Lügen, ...) in visuellen Daten.

Robotik: Hier ist es wieder eine Kombination von Hardware (Elektronik + Mechanik) und Software (Steuerung + Gedächtnis) nötig. Abhängig von den zu erreichenden Aufgaben gibt es unterschiedliche Sensoren (Bewegung, Temperatur, Helligkeit, ...) und Aktuatoren in allen möglichen Ausrichtungen.

Beispiel: Intelligente Prothesen

Spiele: AI wird in der Spieleindustrie schon lange verwendet, z.B. für intelligente Gegner oder intelligentes Softwareverhalten. Auch hier ist es möglich unterschiedliche KI Techniken zu verwenden, der Mensch ist allerdings bei einzelnen Verfahren überlegen.

Ein kombinierter Einsatz von mehreren intelligenten Strategien (Deep Learning) bietet bessere Ergebnisse -> Beispiele: IBM Deep Blue besiegt Schachweltmeister, AlphaGo besiegt zwei Go-Weltklassemaster.

b) *Erklären Sie den Unterschied der Begriffe „künstliche Intelligenz“, „Machine Learning“ und „Deep Learning“.*

Künstliche Intelligenz: Es ist schwer „künstliche Intelligenz“ exakt zu definieren. Ein Beispiel für eine Definition wäre die von McCarthy (1955): „Ziel der KI ist es, Maschinen zu entwickeln, die sich verhalten, als verfügten sie über Intelligenz“.

Machine Learning: Machine Learning ist die Erforschung „intelligenter“ Verfahren, die die Lernfähigkeit (Adaptivität) des Menschen zu imitieren versuchen. Ebenfalls ist es der Versuch, das menschliche Gehirn zu modellieren und zu verstehen.

Deep Learning: Als Deep Learning bezeichnet man das Trainieren von sogenannten „Deep Networks“. Deep Networks sind ein Name für neuronale Netze (=> versuchen das menschliche Gehirn nachzuahmen) mit „vielen“ Schichten (nicht genau definiert wie viele). Mehr als 20 sind üblich.

KI besteht also nicht *nur* aus ML, ML ist nicht *nur* deep learning ($KI \supset ML \supset$ Neuronale Netze $\supset$ Deep Learning).

c) *Worum geht es beim sogenannten Turing-Test?*

Der Turing-Test ist nach Alan Turing benannt. Er war durch seine Definition einer intelligenten Maschine ein KI-Pionier. Allerdings ist diese Definition mehr philosophisch und nicht relevant in der Praxis.

Turing Test: Eine Testperson hat Zugang zu zwei Terminals um zu Kommunizieren – einmal mit einem Menschen und einmal mit einer Maschine. „Eine Maschine kann als intelligent bezeichnet werden, wenn sie die Testperson in 30% der Fälle täuschen kann.“

2. Multiple-Choice (15 Punkte)

Anmerkung: beim Test gibt es nur 10 Fragen.

Kurt Gödel entwickelte neuronale Netze.	☐ richtig	☒ falsch
Der Begriff „Künstliche Intelligenz“ ist eindeutig definiert.	☐ richtig	☒ falsch
Die beste AI-Methode hängt von der Anwendung ab	☒ richtig	☐ falsch
Deep Learning ist für alle Anwendungen die beste Methode.	☐ richtig	☒ falsch
Ein Agent arbeitet ohne Berücksichtigung seiner Umgebung.	☐ richtig	☒ falsch
Ein Agent berücksichtigt seine Umgebung.	☒ richtig	☐ falsch
Logik-Methoden können gut mit Unsicherheit umgehen.	☐ richtig	☒ falsch
Logik ist gut geeignet, wenn Entscheidungen vage sind.	☐ richtig	☒ falsch
Prädikatenlogik ist ausdrucksstärker als Aussagenlogik.	☒ richtig	☐ falsch
Der k-Means-Algorithmus wird für Clustering eingesetzt.	☒ richtig	☐ falsch
Der kNN-Algorithmus wird für Klassifikation eingesetzt.	☒ richtig	☐ falsch
kNN basiert auf neuronalen Netzen.	☐ richtig	☒ falsch
k-Means ist dasselbe wie kNN	☐ richtig	☒ falsch
Ein neuronales Netz kommt ohne Trainingsdaten aus.	☐ richtig	☒ falsch
Neuronale Netze entscheiden immer richtig.	☐ richtig	☒ falsch
„Neuronale Netze“ sind lernfähig.	☒ richtig	☐ falsch
clingo ist ein Programmiersystem für neuronale Netze.	☐ richtig	☒ falsch
Deep Learning verwendet Netze mit vielen Schichten.	☒ richtig	☐ falsch
Überanpassung an das Trainingsset kann auch schlecht sein.	☒ richtig	☐ falsch

3. Logik (12 Punkte)

Angabe

Formalisieren Sie folgende Aussagen als Clingo-Regeln:

- „Zwei Menschen sind Cousins/Cousinen, wenn sie (zumindest) einen gemeinsamen Großelternteil, aber keinen gemeinsamen Elternteil haben.“
- „y ist Tante von x, wenn y weiblich ist und außerdem x ein Kind eines Geschwisternteils von y ist.“
- „Zwei Menschen sind Geschwister, wenn sie *zwei* gemeinsame Elternteile haben.“ (Halbgeschwister zählen somit in diesem Beispiel nicht!)

Verwenden Sie dabei die folgenden Prädikate:

- $\text{human}(x)$: ...x ist ein Mensch
- $\text{male}(x)$: ...x ist männlich
- $\text{female}(x)$: ...x ist weiblich
- $\text{childOf}(y,x)$: ...y ist Kind von x
- $\text{cousin}(y,x)$: ...y ist Cousin bzw. Cousine von x
- $\text{sibling}(y,x)$: ...y und x sind Geschwister
- $\text{uncle}(y,x)$: ...y ist Onkel von x
- $\text{aunt}(y,x)$: ...y ist Tante von x

(Hinweis: Es ist möglich, dass nicht alle Prädikate für die Aufgabe benötigt werden!)

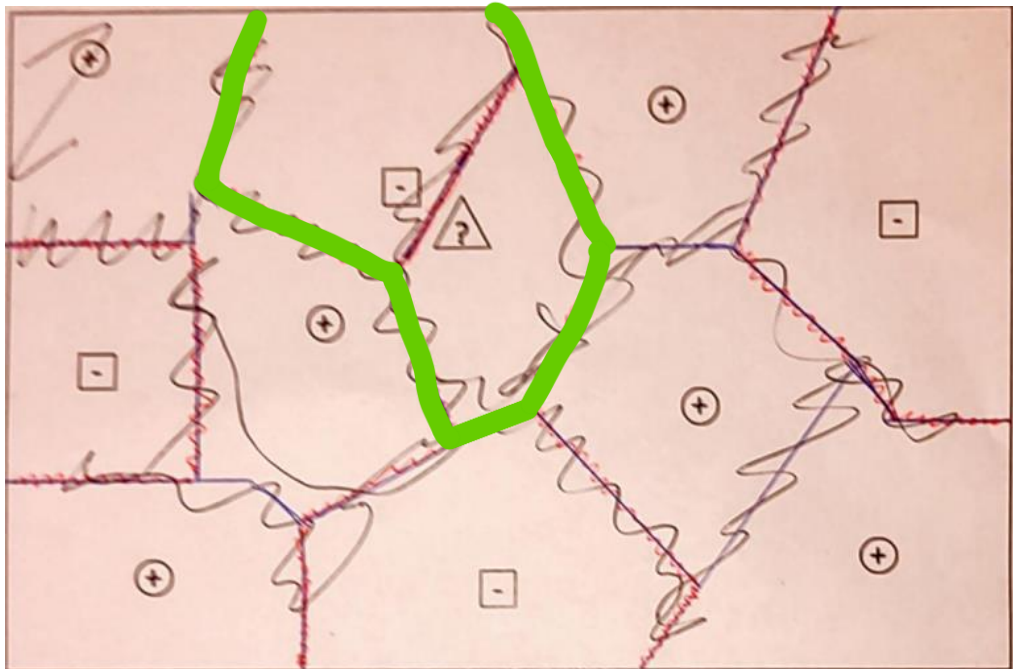
Lösung

1. $\text{cousin}(x,y) :-$
 $\text{childOf}(x,a), \text{childOf}(a,g), \text{childOf}(y,b), \text{childOf}(b,f), \text{sibling}(g,f), g \neq b, \neg \text{sibling}(a,b)$
2. $\text{aunt}(y,x) :- \text{female}(y), \text{childOf}(x,a), y \neq a, x \neq y, \text{sibling}(a,y)$
3. $\text{sibling}(y,x) :- \text{human}(y), \text{human}(x), \text{childOf}(x,p), \text{childOf}(y,p), \text{childOf}(y,q), \text{childOf}(x,q)$

4. Maschinelles Lernen (12 Punkte)

1. Skizzieren Sie für folgende Punktmenge eines $k\text{NN}$ -Modells ($k=1$) das Voronio-Diagramm ein. Die Daten sind als „+“ und „-“ gegeben. (6 Punkte)

Es gibt dabei jeweils die Mitte der Symbole als Punktkoordinate.



2. Wie wird gemäß diesem Modell ($k=1$) der Datenpunkt „?“ klassifiziert? Warum? (3 Punkte)

„?“ wird als „-“ klassifiziert, weil der nächste Nachbar (geringster Abstand – nearest neighbor) ein „-“ ist.

3. *Wie würde der mit „?“ markierte Datenpunkt bei $k=2$ klassifiziert? Warum? (3 Punkte)*

Bei $k=2$ wäre eine Klassifizierung nicht möglich, weil die nächsten zwei Nachbarn verschieden sind (+ & -). Es müsste hier weitere Regeln geben, wenn es zu einem Unentschieden kommt (deswegen sollte k auch nie ein Vielfaches von der Klassenzahl sein).

5. Neuronale Netze (12 Punkte)

a) *Wie ist ein neuronales Netz aufgebaut?*

Ein neuronales Netz versucht, das menschliche Gehirn zu imitieren.

Ein neuronales Netz, hat ein Trainingsset, auf Basis welchem es trainiert wird. Dieses Set muss eine große Menge von Eingabewerten und den dazugehörigen (gewünschten) Ausgabedaten haben.

Durch einen Trainingsalgorithmus wird das Netz dann trainiert und bringt es dazu, zu den Eingaben möglichst die richtigen Ausgaben zu liefern. Das Ziel ist es, dass auch bei neuen Daten die richtige Ausgabe kommt.

Mathematisch: Merkmale sind Eingabevektoren, welche dann vom Neuron mit einem Gewicht versehen werden. Dann werden diese Gewichte summiert und eine Aktivierungsfunktion angewendet. Dann kommt ein Ausgabewert heraus.

b) *Welche Idee steckt hinter dem Konzept Backpropagation?*

Ein neuronales Netz kann inkrementell trainiert werden (es können neue Daten dazu kommen).

Wenn ein Netz mehr als 2 Schichten hat, wird Backpropagation verwendet. Die Idee dahinter ist, dass es in 4 Schritten abläuft:

1. Trainingsmuster anlegen
2. Ausgaben berechnen (Forward-Pass)
3. Trainingsfehler ermitteln (Abweichung vorhergesagte vs korrekte)
4. Backpropagation Fehler zurückspielen und schichtenweise Gewichte anpassen, um Fehler in der Zukunft zu verringern (Backward-Pass)

Gewichtenanpassung sieht man durch den Gradientenabstieg („Abstieg vom Berg (aktueller Fehler) zum minimum“).

c) *Welche Probleme können beim Trainieren von neuronalen Netzen auftreten und welche Möglichkeiten gibt es, diese zu bewältigen?*

Fluch der Dimensionen: geeignete Merkmale werden oft nicht automatisch gefunden. Einfach alle in das verfügbare Netz zu werfen, funktioniert meistens nicht gut, aufgrund von *Redundanzen, irrelevanten Features* -> „Netz sieht vor lauter Bäumen den Wald nicht“.

Lösungen dafür:

- Händische Merkmalsauswahl: Sehr arbeitsaufwändig
- Hauptkomponentenanalyse: Durch Algorithmen werden die Dimensionen reduziert

- Deep Learning: Verwendung von Netzen mit vielen Schichten (Deep Networks) -
> die ersten Schichten sollen die relevanten Fehler finden. Dies ist allerdings
sehr rechenintensiv und benötigt oft teure Hardware.

d) *Welche Aufgaben haben die Faltungs- und Pooling-Schichten in der Topologie eines CNN Netzes?*

In der **Faltungsschicht** wird ein trainierbarer linearer Filter angewendet („Faltung“).

In der **Pooling-Schicht** wird von Eingabeneuronen der Mittelwert oder das Maximum berechnet (-> verwirft die nicht relevanten Informationen).

Fragenkatalog „Einführung AI“

Modul 1: Grundlagen, Ausrichtungen, Anwendungen

1. *Nennen Sie 3 Anwendungen der künstlichen Intelligenz und beschreiben Sie diese kurz.*
[Siehe Hier](#)

2. *Nennen Sie 3 Methoden der künstlichen Intelligenz und beschreiben Sie diese kurz.*

Logik-basierte KI: Basieren auf mathematischer Logik. Es gibt Fakten und Regeln zum Lösen von Problemen (z.B. Mustererkennung, Sprachverarbeitung, semantische Analyse). Anwendungsbereiche: Regelbasiertes Schlussfolgern, regelbasierte Expertensystem (z.B. MYCIN – unterstützendes medizinisches Diagnosetool).

Suche: Es braucht Strategien bei Netzwerken mit vielen Knoten und Übergängen (=sehr viele mögliche Pfade) den „optimalen“ Pfad zu finden (=>“optimal“ wird per Zielsetzung definiert). Anwendungen gibt es z.B. bei Strategiespielen (Schach, Go) oder Ressourcenallokation bei Computernetzwerken.

Maschinelles Lernen & Data Mining: Ist eine „datenbasierte AI“ -> heißt ein Programm lernt aus einer Basis von Daten -> das „erlernte“ Modell kann dann verwendet werden, um Vorhersagen über unbekannte Daten zu erstellen. Voraussetzung hierfür sind „gute“ (=>Größe, Qualität) Datensätze. Anwendungsbereiche: Predictive Analytics, Mustererkennung.

3. *Erklären Sie den Unterschied zwischen den Begriffen „künstliche Intelligenz“, „Machine Learning“ und „Deep Learning“.*

[Siehe Hier](#)

4. *Nennen Sie zwei AI-Methoden und ordnen Sie jeder davon eine typische Anwendung zu, die mit der jeweils anderen Methode nicht gut gelöst werden kann.*

Heuristik: Technik, um ein bestehendes Problem schnellstmöglich zu lösen, wobei die optimale Lösung nicht garantiert sein muss („educated guess“), unter Nutzung von Heuristiken (einfache Regeln). Ansatz: Robotik, Suchmaschinen.

Suche: Sucht die „optimale“ Strategie (je nach Zielsetzung), z.B. bei Schach, Go.

Die Heuristik löst schnell Probleme, während bei der Suche einem „optimalen/perfekten“ Ziel entgegengearbeitet wird.

5. *Worum geht es beim sogenannten Turing-Test?*

[Siehe Hier](#)

Modul 2: Logik

1. Was versteht man in der Logik unter einer Schlussfolgerung?

Eine Schlussfolgerung ist ein logisch zwingender Schluss aus mehreren vorher gelegten Prämissen (bestimmte Aussagen wahr -> damit kommt man zu einer logisch abgeleiteten Aussage). Damit ist sie der abschließende Teil eines Argumentes.

2. Zeigen Sie, dass folgende Formeln Tautologien sind:

1. $\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$

A	B	$\neg A$	$\neg B$	$A \wedge B$	$A \vee B$	$\neg(A \wedge B)$	$\neg A \vee \neg B$	$\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$
0	0	1	1	0	0	1	1	1
0	1	1	0	0	1	1	1	1
1	0	0	1	0	1	1	1	1
1	1	0	0	1	1	0	0	1

2. $A \Rightarrow B \Leftrightarrow \neg B \Rightarrow \neg A$

A	B	$\neg A$	$\neg B$	$A \rightarrow B$	$\neg B \rightarrow \neg A$	$A \rightarrow B \Leftrightarrow \neg B \rightarrow \neg A$
0	0	1	1	1	1	1
0	1	1	0	1	1	1
1	0	0	1	0	0	1
1	1	0	0	1	1	1

3. $((A \Rightarrow B) \wedge (B \Rightarrow A)) \Leftrightarrow (A \Leftrightarrow B)$

A	B	$A \rightarrow B$	$B \rightarrow A$	$(A \rightarrow B) \wedge (B \rightarrow A)$	$A \leftrightarrow B$	$(A \rightarrow B) \wedge (B \rightarrow A) \Leftrightarrow (A \leftrightarrow B)$
0	0	1	1	1	1	1
0	1	1	0	0	0	1
1	0	0	1	0	0	1
1	1	1	1	1	1	1

3. Formalisieren Sie folgende logische Funktionen mit logischen Operatoren und zeigen Sie, dass Ihre Formel korrekt ist:

1. die XOR-Verknüpfung von zwei Variablen

„falschen“ Werte: $(A \vee B)$, $(\neg A \vee \neg B)$ -> verbinden zu einer Konjunktion

A	B	$A \text{ XOR } B$	$(A \vee B)$	$(\neg A \vee \neg B)$	$(A \vee B) \wedge (\neg A \vee \neg B)$
0	0	0	0	1	0
0	1	1	1	1	1
1	0	1	1	1	1
1	1	0	1	0	0

Also ist für $(A \text{ XOR } B)$ die **KNF** = $(A \vee B) \wedge (\neg A \vee \neg B)$

2. die Aussage, dass mindestens zwei von drei Variablen wahr sind (Ergebnis X):

Wir betrachten wieder nur die „falschen“ Fälle (bei denen X falsch ist), also:

- $(A \vee B \vee C)$
- $(A \vee B \vee \neg C)$
- $(A \vee \neg B \vee C)$
- $(\neg A \vee B \vee C)$

Wir verknüpfen die vier Aussagen mit Konjunktion

A	B	C	X	$(A \vee B \vee C)$	$(A \vee B \vee \neg C)$	$(A \vee \neg B \vee C)$	$(\neg A \vee B \vee C)$	$(A \vee B \vee C) \wedge (A \vee B \vee \neg C) \wedge (A \vee \neg B \vee C) \wedge (\neg A \vee B \vee C)$
0	0	0	0	0	1	1	1	0
0	0	1	0	1	0	1	1	0
0	1	0	0	1	1	0	1	0
0	1	1	1	1	1	1	1	1
1	0	0	0	1	1	1	0	0
1	0	1	1	1	1	1	1	1
1	1	0	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1

Deswegen ist für unsere Aussage X die

$$\text{KNF} = (A \vee B \vee C) \wedge (A \vee B \vee \neg C) \wedge (A \vee \neg B \vee C) \wedge (\neg A \vee B \vee C)$$

Stellen Sie das Ergebnis in KNF dar.

4. *Formalisieren Sie folgende Aussagen in der Prädikatenlogik:*

1. **Jeder Mensch hat einen Vater**

$$\forall x \exists y \text{ istVater}(y, x)$$

2. **Manche Menschen haben Kinder**

$$\exists x \exists y \exists z \text{ hatKind}(x, y, z)$$

3. **Alle Vögel fliegen**

$$\forall x \text{ vogel}(x) \rightarrow \text{fliegt}(x)$$

4. **Es gibt ein Tier, das (manche) Körner fressende Tiere frisst**

$$\exists x \exists y \text{ tier}(x) \text{ tier}(y) \text{ frisst}(x, y) \text{ frisstKörner}(y)$$

5. *Schreiben Sie ein Clingo-Programm (vgl. Moodle-Aktivitäten zum Thema Logikprogrammierung) für folgende Aufgabe:*

Die Eingabe sei gegeben durch eine Menge von Fakten der folgenden Art:

- $\text{human}(x)$: ...x ist ein Mensch
- $\text{male}(x)$: ...x ist männlich
- $\text{female}(x)$: ...x ist weiblich
- $\text{childOf}(y, x)$: ...y ist Kind von x

(Es ist möglich, dass nicht alle gegebenen Fakten für die Aufgabe benötigt werden!)

Schreiben Sie Regeln, um folgende Informationen daraus herzuleiten:

$\text{uncle}(y, x)$ für alle Paare von Onkeln y und deren Neffen/Nichten x. (y ist genau dann ein Onkel von x, wenn y männlich ist und außerdem x ein Kind eines Geschwisterteils von y ist.)

- $\text{uncle}(y, x) \text{ :- } \text{male}(y), \text{childOf}(x, z), \text{childOf}(y, a), \text{childOf}(z, a)$

Modul 3: Maschinelles Lernen

1. Erklären Sie den Unterschied zwischen supervised und unsupervised learning

Supervised Learning: Prozess der Erstellung eines Modells auf Basis eines definierten Lerndatensatzes. (**Klassifikation** – Kategoriale Label, **Regression**: Labels sind kontinuierliche Werte).

Unsupervised Learning: Erstellen eines Modells ohne Referenzdatensatzes und mit dem Ziel, Muster zu finden. (**Clustering** – Identifikation von ähnlichen Objekten, **Assoziationsregeln** – finden von zusammengehörigen Objekten).

Beim supervised learning gibt es klassifizierte Trainingsdaten, beim unsupervised learning nicht – dort erkennt es Muster und Ähnlichkeiten.

2. Was versteht man unter Occam's Razor und warum sollte man es anwenden?

Occam's Razor sagt aus, dass man bei gleichwertigen Theorien die einfachere bevorzugen sollte.

Das Ziel ist es in dem Fall, dass mehrere gleich gute (ähnliche Fehlerrate) Theorien existieren, die zu nehmen, die einfacher (kleiner – Entscheidungsbaum) ist.

3. Was versteht man bei Klassifikationsaufgaben unter „false positives“ und „false negatives“?

False Positives: Anteil der fälschlich als positiv klassifizierten Objekten an der Gesamtheit der tatsächlich negativen Objekte.

False Negatives: Anteil der fälschlich als negativ klassifizierten Objekte an der Gesamtheit der tatsächlich positiven Objekte.

4. k-Nearest Neighbor

- a. Skizzieren Sie für folgende Punktmenge eines kNN-Modells ($k=1$) das Voronoi Diagramm



- b. Wie würde ein neuer Datenpunkt in der rechten unteren Ecke des klassifiziert werden?

Der neue Datenpunkt würde als „+“ klassifiziert werden, da der nächste Nachbar ($k=1$) ein „+“ ist.

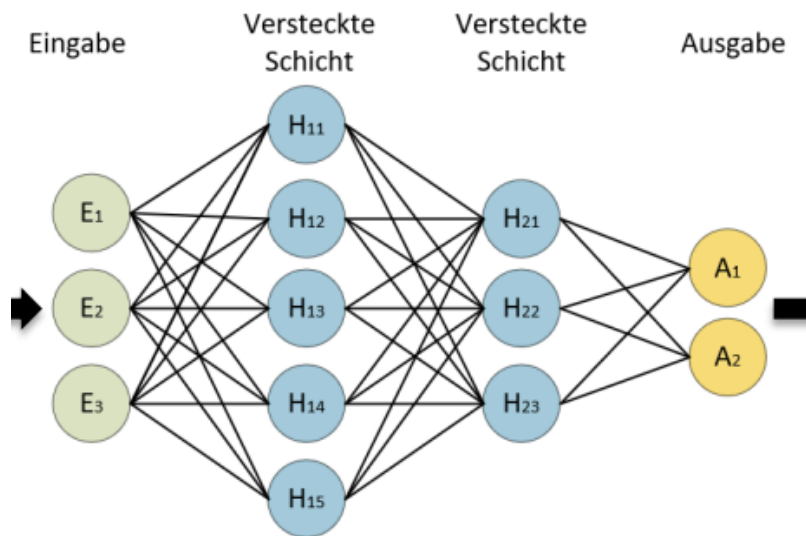
- c. Wie würde sich die Klassifikation bei einem Wechsel auf $k=3$ ändern?

Bei einem Wechsel auf $k=3$ würde die Klassifikation auf „-“ wechseln, da 2 von 3 nächsten Nachbarn ein „-“ sind.

5. Gegeben sind die folgenden Punkte im 2-dimensionalen Raum: $p_1(40/53)$, $p_2(22/38)$, $p_4(26/19)$, $p_5(8/41)$, $p_6(45/30)$
- Berechnen Sie die euklidischen Distanzen zwischen den einzelnen Punkten.
 - Wenden Sie das k-Means mit $k=2$ unter Verwendung der euklidischen Distanz an.

Modul 4: Neuronale Netzwerke

1. Skizzieren Sie ein neuronales Netz und benennen Sie dessen Elemente



[Siehe Auch Hier](#)

2. Wie funktioniert das Trainieren eines neuronalen Netzes mittels Backpropagation? (Es sind keine Formeln notwendig, die Erklärung der Idee reicht)

[Siehe Hier](#)

3. Was sind Deep Neural Networks?

Deep Neural Networks sind Sammlungen von neuronalen Netzen, welche mithilfe von Deep Learning trainiert werden. Hier gibt es „viele“ Schichten (keine einheitliche Definition, ab wann ein Neural Network als Deep Network zählt, aber üblich sind 20+).

Je nach Typen der Schichten gibt es hier weitere Unterscheidungen, u.a. in:

- Convolutional Neural Networks (CNNs)
- Deep Belief Networks (DBNs)
- Recurrent Neural Networks (RNNs)

4. Was sind Convolutional Neural Networks (CNNs)?

Hier läuft das unattended Learning über alle Merkmale (=sehr aufwändig).

Merkmalschichten sind auch nicht vollständig, sondern nur über wenige Neuronen mit der vorherigen Schicht vernetzt. Bei einem CNN gibt es beim Lernen abwechseln Faltungs- (convolution) und Pooling-Schichten:

- In der Faltungsschicht wird ein trainierbarer linearer Filter angewendet
- In der Pooling-Schicht wird von Eingabeneuronen der Mittelwert oder das Maximum berechnet

5. *Erklären Sie die Grundidee der Funktionsweise von ChatGPT und ähnlichen Systemen.*

Die Grundidee von ChatGPT (und ähnlichen LLMs = Large Language Models) ist das Vorhersagen der wahrscheinlichsten Antwort („Vervollständigung“ eines Textes durch das wahrscheinlichste nächste Wort bzw. Token). Die Trainingsdaten dafür sind Milliarden Websites (inkl. Wikipedia) und Bücher.

Vereinfacht gesagt sieht ChatGPT den gesamten Chatverlauf als den zu vervollständigenden Text.

GPT steht für „Generative Pretrainer Transformer“, eine spezielle Form von Deep Neural Networks.

Probleme hierbei: Die Antworten sind nicht zwingend richtig, auch wenn sie sich evt. Gut anhören.

Clingo Aufgaben

Schengen Countries 1

Input

Assume that the input is a set of facts specified by the following predicates:

- `adjacent(X, Y)`: country X is adjacent to country Y
- `eu(X)`: country X is a member of the European Union
- `candidate(X)`: country X is a candidate member
- `euro(X)`: country X uses the Euro
- `schengen(X)`: country X belongs to the Schengen area

Note: The name of a state is given by a string constant.

Task

Write an ASP program that computes all Schengen countries that share a border with a Non-Schengen country. For each such county C, derive an atom `outerBorder(C)`.

Solution

`outerBorder(C) :- schengen(C), adjacent(C, Y), not schengen(Y).`

Schengen Countries 2

Input

Assume that the input is a set of facts specified by the following predicates:

- `adjacent(X, Y)`: country X is adjacent to country Y
- `eu(X)`: country X is a member of the European Union
- `candidate(X)`: country X is a candidate member
- `euro(X)`: country X uses the Euro
- `schengen(X)`: country X belongs to the Schengen area

Task

Write an ASP program that computes all EU countries that currently share a border with a non-Schengen country, but would not share such a border anymore after all current EU and EU candidate countries join the Schengen area. For each such county C, derive an atom result(C).

(In this example "outer border" is strictly defined by EU-membership: borders with countries like VaticanCity or Switzerland count as outer borders, although they are often not called that way!)

You can (but do not have to) use helper predicates, but make sure to keep the #show directives in the answer box!

Solution

```
non_schengen_neighbor(X,Y) :- adjacent(X, Y), eu(X), not schengen(Y).
future_schengen(X) :- eu(X).
future_schengen(X) :- candidate(X).
future_non_schengen_neighbor(X, Y) :- adjacent(X,Y), eu(X), not future_schengen(Y).
result(C) :- non_schengen_neighbor(C,_), not future_non_schengen_neighbor(C, _).
```

Birds

Input

Assume that the input is given by the following predicates:

- eagle(X) specifies that X is an eagle
- penguin(X) specifies that X is a penguin

Task

Write an ASP program to represent the following knowledge:

- eagles are birds
- penguins are birds
- birds can fly, except for penguins, who cannot fly

Use the following predicates:

- bird(X) specifies that X is a bird
- fly(X) specifies that X can fly
- -fly(X) specifies that X cannot fly

Solution

```
bird(X) :- eagle(X).
bird(X) :- penguin(X).
fly(X) :- bird(X), not penguin(X).
-fly(X) :- penguin(X).
```

Cousin

Input

Assume that the input is a set of facts specified by the following predicates:

- childOf(X, Y): X is a child of Y
- female(X): X is female
- male(X): X is male

Task

Write an ASP program that derives atoms over the following predicates:

- `cousin(X, Y)`: whenever X is a cousin of Y

Solution

`sibling(X,Y) :- childOf(X, P), childOf(Y, P), X != Y.`

`cousin(X,Y) :- childOf(X,PX),childOf(Y,PY),sibling(PX, PY),X != Y.`

Adjacent States

Input

Assume that the input is a set of facts specified by the following predicates:

- `state(X)`: X is a US state
- `adjacent(X, Y)`: state X is adjacent to state Y

Note: The name of a state is given by a constant starting with lower-case letter (without quotes). If the state name consists of multiple words, they are concatenated without blanks and each new word (except the first one) starts with an upper-case letter. Examples: `california`, `florida`, `newYork`, `northDakota`, `rhodelsland`, `districtOfColumbia`.

Task:

Write an ASP program that computes in predicate `result(.)` the US states that are adjacent to at least five other states.

Solution

`countAdjacent(State, Count) :- state(State),Count = #count{ Y : adjacent(State, Y) }.`

`result(State) :- state(State),countAdjacent(State, Count),Count >= 5.`

Public Transport 1

Input

The input defines the connections between two stations in the public transport system in Vienna. It is a set of facts of form

`conn(From, To, Type, Line).`

where `From` and `To` are quoted station names, `Type` declares the type of transportation (bus, tram, train or metro; all without quotes), and `Line` specifies the quoted name of the line.

Examples:

- `conn("Vorgartenstraße","Donauinsel",metro,"U1").`
- `conn("Siedlung Sonnheim","Eisstößgasse",bus,"93A").`
- `conn("Aumannplatz","Weinhauser Gasse",tram,"41").`
- `conn("Laa an der Thaya Bahnhof","Kottingneusiedl Bahnhof",train,"S2").`

Moreover, assume that a fact of form `start(S).`, where S is a station is, is given.

Task

Write an ASP program that computes all nodes reachable from S by metro only (but with an arbitrary number of changes). The result is to be encoded using atoms of form `reachable(.)`.

Solution

reachable(S) :- start(S).

reachable(To) :- conn(From, To, metro, Line), reachable(From).

Public Transport 2

Input

The input defines the connections between two stations in the public transport system in Vienna. It is a set of facts of form

`conn(From, To, Type, Line).`

where From and To are quoted station names, Type declares the type of transportation (bus, tram, train or metro; all without quotes), and Line specifies the quoted name of the line.

Examples:

- `conn("Vorgartenstraße","Donauinsel",metro,"U1").`
- `conn("Siedlung Sonnheim","Eisstößgasse",bus,"93A").`
- `conn("Aumannplatz","Weinhauser Gasse",tram,"41").`
- `conn("Laa an der Thaya Bahnhof","Kottingneusiedl Bahnhof",train,"S2").`

Moreover, assume that a fact of form `start(S).`, where S is a station is, is given.

Task

Write an ASP program that computes all nodes reachable from S by metro with arbitrary many line changes but at most three stops (excluding S but including the destination). The result is to be encoded using atoms of form `reachable(.)`.

Solution

reachable(S, 0) :- start(S).

reachable(To, N+1) :- reachable(From, N), conn(From, To, metro, Line), N < 3.

reachable(To) :- reachable(To, N), N > 0.

Father/Mother

Input

Assume that the input is a set of facts specified by the following predicates:

- `childOf(X, Y)`: X is a child of Y
- `female(X)`: X is female
- `male(X)`: X is male

Task

Write an ASP program that derives atoms over the following predicates:

- `father(X, Y)`: whenever X is the father of Y
- `mother(X, Y)`: whenever X is the mother of Y

Solution

father(X,Y) :- male(X),childOf(Y,X).

mother(X,Y) :- female(X),childOf(Y,X).

Missouri

Input

Assume that the input is a set of facts specified by the following predicates:

- `state(X)`: X is a US state
- `adjacent(X, Y)`: state X is adjacent to state Y

Note: The name of a state is given by a constant starting with lower-case letter (without quotes). If the state name consists of multiple words, they are concatenated without blanks and each new word (except the first one) starts with an upper-case letter. Examples: `california`, `florida`, `newYork`, `northDakota`, `rhodeIsland`, `districtOfColumbia`.

Task

Write an ASP program that computes in predicate `result(X)` the US states that can be reached from `missouri` over exactly one other state (but not directly).

Solution

`reachable_1(S) :- adjacent(missouri, S), state(S).`

`result(S) :- adjacent(X, S), reachable_1(X), state(S), not reachable_1(S), S != missouri.`

Nodes

Input

The input defines a graph as follows:

- `node(X)` defines that X is a node
- `arc(X, Y)` defines that there is a (directed) arc from X to Y

Task

Write an ASP program that computes all nodes X that have both outgoing and incoming arcs. The result is to be encoded using atoms of form `output(.)`.

Solution

`output(N) :- node(N), arc(N, X), arc(Y, N).`